

RFC-001 · Version 3

Permission-Aware Enterprise Knowledge Platform

Palantir-class semantics. Open to everyone.

Engineering RFC

May 2026 · Draft for review

Author: Platform team · Status: open for comment

Front matter

RFC	001 v3 (supersedes v1, v2)
Status	Draft, open for review
Date	May 2026
Length	Comprehensive technical implementation manual
Scope	Connector framework, ingestion modalities, layered access control, bitemporal RDF knowledge graph, ontology + fact-mining agents, MCP server/host, retrieval pipeline, actions, deployment topology, security, scale, milestones.
Primary competitors	Palantir Foundry, Glean, Microsoft 365 Copilot, Gemini Enterprise, Amazon Q Business, Dust, Coworker AI, DevRev Computer, GoSearch, Notion AI, Onyx, Guru, Zep / Graphiti.
Inspirations	Palantir Ontology (objects + links + actions + dynamic security); Glean (zero-copy ACL inheritance); Zep / Graphiti (bitemporal facts); Microsoft Fabric / OneLake (authorized-engine model, sensitivity labels); Snowflake / Databricks (row + column + cell policies); Airbyte / Nango / Fivetran (connector taxonomy); Debezium (CDC); Zanzibar / SpiceDB / OpenFGA (ReBAC); W3C PROV (provenance).
Standards used	RDF 1.1, OWL 2, SHACL, SPARQL 1.1, BiTRDF (bitemporal RDF), W3C PROV-O, Model Context Protocol, OpenLineage, OAuth 2.1.
Authorization core	SpiceDB (primary), OpenFGA (alternative).
Vector store	pgvector default; Qdrant / Turbopuffer at scale.
Triple store	GraphDB (free tier dev), Stardog or Jena Fuseki prod.

Contents

1. Executive summary
2. Change log vs. v2
3. Vision: a Palantir-class platform, open to everyone
4. Competitive landscape (deep, current)
5. System overview
6. Composability principles
7. Connector framework
8. Ingestion modalities
9. Connector reference (per-source detail)
10. Permission model: unified principals
11. Layered access control (object, row, column, cell, label)
12. Bitemporal knowledge graph
13. Ontology layer (OWL + SHACL)
14. Ontology agents
15. Fact-mining agents
16. Entity resolution
17. Provenance, lineage, audit (W3C PROV)
18. Retrieval pipeline (hybrid)
19. Actions: the kinetic layer
20. MCP integration
21. Reference deployment topology
22. Security model
23. Performance, scale, SLAs
24. Build vs. buy decisions
25. Milestones
26. Open questions
27. References

1. Executive summary

This document specifies an open, permission-aware enterprise knowledge platform: a system that ingests heterogeneous source data from across an organization, mirrors source access controls into a unified authorization layer, materializes facts into a bitemporal knowledge graph governed by a customer-evolved ontology, and exposes the whole platform through the Model Context Protocol (MCP) so that any MCP-aware client — Cursor, Claude Code, Claude Desktop, ChatGPT, Glean, custom agents — can use it as their substrate.

What we are building, in one sentence. Palantir Foundry's ontology, kinetic actions, and dynamic security model — open, MCP-native, and available to every company, not just defense and Fortune-50 customers paying eight-figure ACVs.

Why this and not naive RAG. Three architectural commitments separate this from every vector-only product on the market:

- **Semantic + kinetic, not just retrieval.** Following Palantir, the platform represents not only data (objects, properties, links) but also the operations performed on that data (actions, functions). Actions carry validation, audit, and write-back to source systems. Read-only semantic layers are a strict subset of what we ship.
- **Bitemporal facts.** Every fact carries both valid time (when it was true in the world) and transaction time (when we learned it). Borrowed from Zep / Graphiti and the BiTRDF model. This is the only honest way to answer "who owned this account when the deal closed" — a question every enterprise will ask.
- **Standards-based interop.** RDF, OWL, SHACL, SPARQL — the W3C stack. Glean's knowledge graph is proprietary; ours is not. Customers can dump their facts as RDF, query with SPARQL, validate with SHACL, and run W3C-PROV provenance queries with no vendor lock-in. This is a deliberate moat against the closed competitors.

Why the moat is permissioning, even now. Every competitor has shipped some version of RAG by mid-2026. Far fewer have shipped honest, source-of-truth, real-time ACL enforcement that survives security review at a Fortune-500. The detailed per-source extraction work in §9 is what is hard to replicate. Glean's \$200M ARR and \$7.2B valuation in 2025 was earned on this exact dimension — and Microsoft Copilot, Google Gemini Enterprise, and Amazon Q are each strong only inside their own ecosystem. The cross-ecosystem, permission-aware, agent-substrate position is wide open.

Document scope. This is the master technical implementation manual. It covers connector taxonomy, multiple ingestion modalities, layered access control (down to cell-level), bitemporal RDF graph design, ontology and fact-mining agents, the retrieval pipeline, the kinetic action layer, MCP server and host, reference deployment topology, security model, scale targets, and a build vs. buy decision matrix for every major component.

2. Change log vs. v2

- **New: Palantir-class scope.** v3 explicitly targets the Palantir Foundry feature surface — semantic objects, kinetic actions, dynamic security — rather than positioning as just an ACL-aware RAG.
- **New: Composability principles (§6).** An explicit set of design rules so the platform stays simple to assemble and reason about.

- **New: Connector taxonomy (§7).** Three connector tiers, four ingestion modalities (poll, webhook, CDC, MCP-host, file-drop), and a clean SDK contract per tier.
- **New: 15 source-by-source ACL extraction guides (§9).** Google Drive, Confluence, Slack, Notion, SharePoint, Jira, Salesforce, GitHub, Gmail, Box, Dropbox, ServiceNow, Zendesk, Workday, Linear, plus generic patterns for SQL warehouses, S3-style object stores, and HTTP webhook sources.
- **New: Layered access control (§11).** Object-level, row-level, column-level, cell-level, sensitivity labels, and dynamic policies — explicitly inspired by Snowflake, Databricks, OneLake, and Palantir.
- **New: Entity resolution (§16).** Cascade matching (rule → ML → LLM), transitive-closure safeguards, semantic blocking.
- **New: Actions / kinetic layer (§19).** Write-back operations with validation, idempotency, audit trail, and approval workflows. The thing Palantir built and Glean is still catching up to.
- **New: Reference deployment topology (§21).** Single-tenant SaaS, multi-tenant SaaS, customer-VPC, fully self-hosted. Trade-offs spelled out.
- **New: Build vs. buy matrix (§24).** Component-by-component recommendation: what to write, what to embed open-source, what to license from a vendor.
- **Expanded: every other section.** Deeper, with code, schemas, and per-source detail.

3. Vision: a Palantir-class platform, open to everyone

Palantir Foundry is the most architecturally complete commercial enterprise data platform. Its core abstraction — the Ontology — is not just a schema or a semantic layer. It is what Palantir calls a fourfold integration: data, logic, action, and security, woven into a single operational substrate. Objects (nouns) describe what exists; actions (verbs) describe what can be done; functions encode arbitrary business logic; dynamic security enforces who can read or write what at every level.

Foundry's commercial success is real — multi-billion-dollar revenue, embedded in defense and intelligence and increasingly in Fortune-100 manufacturing and finance. Its limitations are also real: it is proprietary by design, ingestion-based with custom APIs, and deployments are service-intensive (often running to seven or eight figures of total cost of ownership). Customers benefit from a highly integrated ontology but lose flexibility, openness, and direct access to their own semantic models.

This RFC proposes the converse positioning:

- **Equal architectural depth.** Ontology with objects, links, actions, functions, and dynamic security — the full Palantir feature surface. Plus bitemporal facts, which Foundry does not natively offer.
- **Open by default.** RDF, SHACL, SPARQL, PROV-O. The customer's graph is their graph; they can export, federate, or replace us without re-modeling. This eliminates the lock-in argument that is Palantir's single largest objection in procurement.
- **MCP-native.** The platform is reachable from every MCP-aware tool on day one. Cursor, Claude Code, Claude Desktop, ChatGPT, Glean, and custom agents all consume the same surface.
- **Self-serve onboarding.** Zero forward-deployed engineer requirement. Connectors are one-click OAuth; ontology agents propose the schema from the customer's actual data; SHACL shapes evolve automatically with human approval gates.
- **Tiered pricing, including a free tier.** The core platform is open source; managed cloud is the commercial offering. This is what lets us be "Palantir for everyone" rather than "Palantir for the people who can afford Palantir".

Slogan. Palantir's ontology, Glean's ACL discipline, Zep's bitemporal honesty, MCP's universality. Open source core.

4. Competitive landscape (deep, current)

The market has consolidated into four bands. Where products sit determines what they can credibly do, and where they will get boxed in. We are positioned in the fourth band — explicitly.

4.1 Band 1: Operational platforms (Palantir)

Palantir Foundry. The architectural reference. Ontology as a fourfold integration (data, logic, action, security). Microservice backend (Object Storage V2 / Funnel / OMS). High-fidelity write-back to source systems via Change Data Capture. Branching and proposal-based governance for ontology changes — exactly the pattern we adopt for ontology agents in §14. Proprietary, expensive, service-heavy. Strongest in defense, intelligence, manufacturing, supply chain.

4.2 Band 2: Enterprise search / agentic platforms

Glean

\$200M ARR, \$7.2B valuation, doubled ARR in nine months in 2025. Repositioned in early 2026 as the "intelligence layer beneath the interface" (TechCrunch, Feb 2026). Connects 100+ sources, zero-copy indexing, deep ACL inheritance, Enterprise Knowledge Graph that maps users / groups / docs / activity. Agentic Engine 2 (parallel sub-agent orchestration), 15-LLM model hub, MCP server AND host, Glean Protect Plus for governance. The closest commercial analog to what we are building, but with proprietary graph and no bitemporal layer.

Microsoft 365 Copilot

The default for Microsoft-heavy enterprises. Native Entra ID permissions, deep Microsoft Graph integration, Copilot Studio for custom agents, +30/user/month on top of M365 licenses. Boxed in by ecosystem — limited reach into Salesforce, Atlassian, Google.

Gemini Enterprise

The Google-Workspace analog. Domain-wide delegation for Drive permissions, 1M-token context, strong document summarization. Same single-ecosystem limitation as Copilot.

Amazon Q Business

Aggressively priced (\$20/user/month, \$3/user/month Lite). IAM Identity Center inheritance, 40+ datasources. Best fit for AWS-heavy enterprises. Less polished UX, less workflow execution.

Dust

Open-source (MIT), agent-first. 100+ integrations plus MCP. Per-agent fine-grained permissions. The closest open-source competitor in spirit, but does not yet have a knowledge graph or bitemporal facts.

Coworker AI

Workflow-centric. "OM1 / Organizational Memory." Read+write surface (updates Salesforce, files Jira tickets). Flat \$30/user/month. 40+ connectors. Less published architectural detail.

DevRev Computer, GoSearch, Onyx, Guru, Notion AI

DevRev positions explicitly against Glean ("Glean searches; Computer acts"). GoSearch is price-aggressive with hybrid indexed+federated. Onyx (Danswer) is the open-source self-hostable search incumbent. Guru is governance-first / verified-knowledge. Notion AI is the lowest-friction Notion-only play.

4.3 Band 3: Memory infrastructure

Zep / Graphiti

Not enterprise search, but the most important architectural reference for our bitemporal layer. Graphiti is open-source (Apache, Neo4j-backed) and implements a three-subgraph hierarchy (episode, semantic entity, community) with bitemporal edges. Beats MemGPT on the DMR benchmark (94.8% vs 93.4%). Scales to 100k-token conversations. We adopt the model, not the implementation.

Microsoft GraphRAG

Query-Focused Summarization over an LLM-extracted knowledge graph. Strong for global summaries ("what are the main themes") but batch-indexed, less suited to the frequently-updated enterprise case Graphiti and we target. Useful as a community-detection reference for §12.

LightRAG, Mem0, Letta (MemGPT)

LightRAG (EMNLP 2025) is an interesting middle ground — a fast, graph-augmented RAG that uses entity-relationship extraction without the full ontology overhead. Mem0 and Letta are single-agent memory libraries. We borrow extraction patterns but architect at platform level, not library level.

4.4 Band 4: Open knowledge platform (our position)

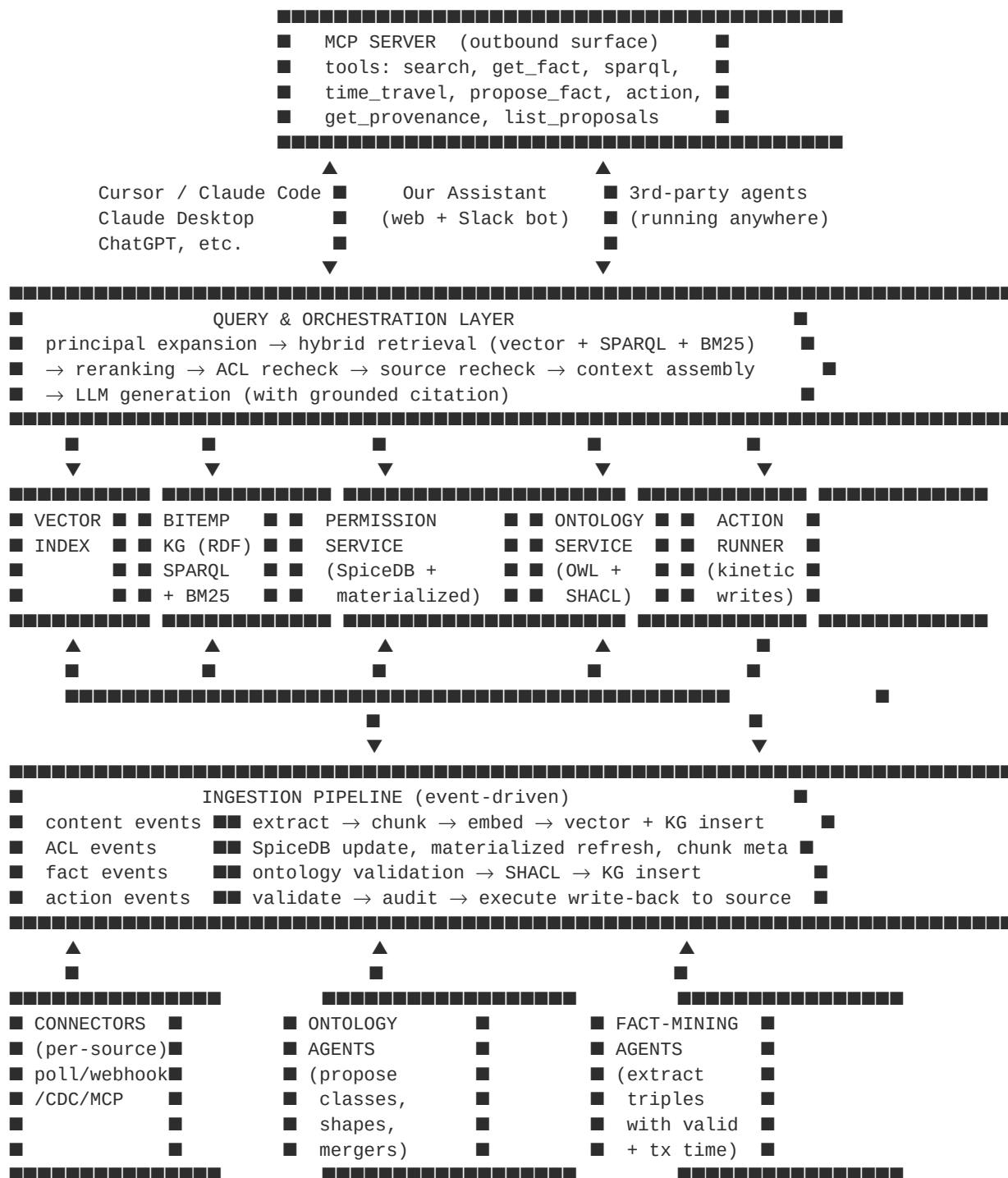
Nothing in the market today combines all of: Palantir's ontology depth, Glean's ACL discipline, Graphiti's bitemporality, an open standards-based semantic stack, and MCP-native distribution. The gap is real, and 2026 is the right year for it. Competitors who try to retrofit any one of these to their existing stack will burn 18+ months.

4.5 Feature comparison

Product	Ontology	Bitemporal	Open standards	Cross-source ACL	Actions	MCP	Open core
Palantir Foundry	Yes (proprietary)	Partial	No	Yes (proprietary)	Yes (rich)	Limited	No
Glean	Knowledge graph (proprietary)	Partial	No	Yes	Yes (Agentic Engine 2)	Server + host	No
M365 Copilot	MS Graph	No	No	Entra-native	Yes (within M365)	Limited	No
Gemini Enterprise	Limited	No	No	Drive-native	Yes (within GWS)	Limited	No
Amazon Q	Limited	No	No	IAM IC	Yes	Limited	No
Dust	Limited	No	No	Per-agent + source	Yes	Yes	Yes (MIT)
Coworker AI	OM1 memory	Unclear	No	Source-inherent	Yes (CRUD)	Unclear	No
DevRev Computer	Computer Memory	Unclear	No	Source + RBAC	Yes (CRUD)	Unclear	No
Zep / Graphiti	Three-subgraph	Yes	No	N/A (memory)	N/A	Yes	Yes (Apache)
This RFC	Full (OWL+S HACL)	Yes	Yes (RDF/SH ACL/SPARQL/PROV)	Yes (Zanzibar)	Yes	Server + host	Yes

5. System overview

Nine services. Each has a single responsibility. The first four are infrastructure; the next three are domain logic; the last two are the integration surface.



5.1 Service responsibilities

Service	Responsibility
Connectors	Per-source data + ACL extraction. Common SDK interface. Tiered (managed, community, custom).
Ingestion pipeline	Event-driven. Routes content, ACL, fact, and action events to downstream consumers. Kafka or equivalent.
Permission service	SpiceDB live decisions + Postgres materialized ACLs + bi-temporal audit log.
Bitemporal KG	RDF triple store with valid + transaction time on every fact. SPARQL endpoint. SHACL validation on write.
Vector index	Embedding store with metadata pre-filter (principals, source, time). pgvector or Qdrant.
Ontology service	OWL + SHACL store. Versioned in Git. Customer-readable, Protégé-compatible.
Action runner	Validated, audited write-back to source systems. Idempotent. Approval workflows.
Query layer	Hybrid retrieval (vector + SPARQL + BM25), reranking, ACL recheck, context assembly, LLM call.
MCP server / host	Outbound: tools for any MCP client. Inbound: consume source-system MCP servers (Supabase, GitHub, etc.) where they exist.

6. Composability principles

Five rules. They are not optional. Every architectural decision is checked against them.

6.1 One responsibility per service

The permission service does authorization. It does not index content, run agents, or store facts. The KG stores facts; it does not enforce permissions (it pre-filters with denormalized ACLs but the authoritative check is always SpiceDB). The action runner executes writes; it does not validate ontology — it asks the ontology service. This is what makes the system debuggable: every incident has a clear owner.

6.2 Events as the integration substrate

All cross-service communication is via durable, replayable event streams (Kafka or equivalent). No service calls another service synchronously to perform a write. This is what makes the system operable: a backed-up consumer never blocks producers, and a bad deployment is recoverable by rewinding the stream.

6.3 Open standards at every boundary

RDF + SHACL + SPARQL for the graph. OAuth 2.1 + OIDC for identity. MCP for agent integration. W3C PROV-O for provenance. OpenLineage for pipeline metadata. The platform's external surfaces are all standards-based; proprietary innovation lives strictly inside the services that benefit from it (e.g. our hybrid retrieval ranker).

6.4 The connector SDK is the contract

Every connector — whether built by us, the community, or the customer — implements the same SDK interface. New connectors do not need code changes to the core. This is what makes the system extensible: a customer with an obscure internal tool ships a connector in a day.

6.5 Human review is a first-class workflow, not a hack

Ontology changes, low-confidence facts, sensitive actions: all flow through an approval queue with full provenance. This is borrowed directly from Palantir's branch-and-proposal pattern, and it is what makes the system trustworthy enough to act on real operational data.

7. Connector framework

The connector layer is the part of the platform that touches the most surface area and consequently is where most engineering hours go in the first 18 months. Every product in this space — Airbyte, Fivetran, Nango, Glean, Palantir — has converged on a similar architectural answer: a tiered library, a stable SDK, and a uniform contract for content + permissions + change feeds.

7.1 Tiered connector library

Tier	Built by	SLA	Examples
Managed	Us	Full SLA, dedicated maintenance, 24/7 on-call. OAuth managed end-to-end.	Google Drive, Confluence, Slack, Notion, SharePoint, Jira, Salesforce, GitHub, Gmail, Outlook
Verified	Community + us	Tested, code-reviewed, semi-supported. Best-effort.	Box, Dropbox, Zendesk, ServiceNow, Workday, Linear, Asana, Monday, Coda, Airtable, HubSpot, Tableau
Custom	Customer	Customer-supported. Same SDK contract — they get docs, examples, and a test harness.	Proprietary internal tools, niche SaaS, on-prem databases

Airbyte runs roughly the same tiering (Airbyte Managed / Marketplace / Custom) and reports ~15% of source connectors are Managed-tier as of mid-2025. We will target ~25-30 Managed connectors at GA — the ones that cover 80% of every enterprise's surface area — and rely on the SDK to make Verified and Custom viable for the long tail.

7.2 The Connector SDK

Every connector implements one interface. The interface is small by design — no "convenience" methods that vary between implementations.

[illegible]

7.3 What every connector must produce

- **Content events.** Document added / updated / deleted with stable IDs.
- **ACL events.** Document permission changes, group membership changes, deletion-of-access.
- **Schema metadata.** Source-native types (a Salesforce Account vs. a Notion page vs. a Slack message) get registered as upper-ontology object types automatically.
- **Action descriptors.** If the connector supports writes, what actions are available ("create Jira ticket", "update Salesforce Opportunity stage", "send Slack message"), with JSON Schema input validation and required permission scopes.

8. Ingestion modalities

Five distinct shapes for getting data in. Connectors declare which they support; many support more than one (e.g. webhook + periodic poll as a fallback).

8.1 Modality 1 — Pull / Poll

Simplest. Connector periodically calls the source API for new or changed items, paging through with a stable cursor. Works for any source with a REST or GraphQL API. Latency is bounded by poll frequency. Used as the fallback when webhooks or CDC are unavailable.

```
async function pollLoop(conn: Connector) {
  let cursor = await store.loadCursor(conn.id) ?? 'beginning'
  while (true) {
    const { changes, nextCursor } = await conn.getChangesSince(cursor)
    for (const c of changes) await events.publish(c)
    await store.saveCursor(conn.id, nextCursor)
    await sleep(pollInterval(conn)) // 30s to 15m based on source
  }
}
```

8.2 Modality 2 — Webhook

Source pushes events to us. Lowest latency for content changes. Required for Slack, GitHub, Salesforce, most SaaS. Sources that support content webhooks but not permission webhooks (Confluence) are paired with a nightly ACL resync.

```
// Each webhook endpoint maps source events to our ChangeEvent schema.
app.post('/wh/slack/:tenant', verifySig, async (req, res) => {
  const ev = mapSlackEvent(req.body)
  await events.publish({ tenant: req.params.tenant, ...ev })
  res.status(204).end()
})
// We always 2xx fast and process async – the source retries on 5xx.
// Idempotency: every ChangeEvent carries a source_event_id that
// downstream consumers dedupe on.
```

8.3 Modality 3 — Change Data Capture (CDC)

For SQL warehouses, on-prem databases, and any source where we have log-level access. Debezium is the open-source standard: connectors for PostgreSQL, MySQL, MongoDB, SQL Server, Oracle, and more, publishing row-level change events to Kafka without polling. This is how Palantir Foundry achieves "extremely low-latency mirroring" — they say CDC explicitly in their architecture docs.

- **When to use:** customer's data warehouse (Snowflake, BigQuery, Redshift, Databricks), customer's transactional database, or any source where a Debezium connector exists.
- **Why log-based and not query-based:** log-based has minimal impact on source load, captures the exact order of operations, and doesn't miss deletes (which query-based CDC does).
- **Permissioning caveat:** CDC gives us the rows but not necessarily the row-level policies that gate them in the source. For Snowflake / Databricks / Redshift, we read row-access policies and column-mask policies separately via metadata catalogs and translate them into the unified principal model.

8.4 Modality 4 — MCP-host (consume source MCP servers)

Increasingly common in 2026: many SaaS products ship their own MCP server (Supabase, GitHub, Linear, Notion, Airbyte itself is now positioned as an MCP-served "Context Layer"). Our platform can consume these directly, dramatically shortening time-to-integration.

Important caveat: source MCP servers generally do NOT expose permission introspection. They expose tools, not ACLs. So MCP-host ingestion is only viable for content; we still need a separate channel (REST API, admin API) to extract permissions. We treat the MCP server as a fast content fetch and the admin API as the ACL source.

8.5 Modality 5 — Object store / file drop

For bulk historical loads, third-party data feeds, and the "please ingest this S3 bucket" use case. The platform watches an S3-compatible path (S3, GCS, Azure Blob, OneLake), parses files according to detected format, and routes them through the standard ingestion pipeline. Format support at v1: CSV, JSON / NDJSON, Parquet, Avro, plain text, PDF, DOCX, PPTX, XLSX, EML / MBOX.

Permissions for file-drop ingestion come from sidecar metadata (a JSON file alongside the data describing principals) or, for OneLake / Fabric customers, from native OneLake roles via the authorized-engine model. Where neither is available, the customer explicitly assigns an ACL at the path level.

8.6 Modality selection table

Source class	Recommended	Fallback	Notes
SaaS with rich webhooks (Slack, GitHub, Salesforce)	Webhook	Poll	Webhook for content; poll for ACL diffs.
SaaS with weak webhooks (Confluence, Notion)	Webhook + nightly resync	Poll	Page-restriction late-binding for high-sensitivity tenants.
SaaS with strong delta API (Drive, SharePoint)	Poll (changes.list / delta)	Webhook for content	changes.list with permission expansion.
Data warehouse (Snowflake, BigQuery, Redshift, Databricks)	CDC or scheduled extract	Native streaming	Read row + column policies separately.
Transactional DB (Postgres, MySQL, MongoDB)	CDC (Debezium)	Read-replica scan	Schema evolution events are first-class.
Object store (S3, GCS, OneLake)	File-drop watcher	Periodic scan	Sidecar ACL metadata.
MCP-enabled SaaS	MCP-host for content	REST for ACL	Treat as fast content channel; pair with admin API for ACLs.
Custom internal tools	Custom connector via SDK	S3 export	Customer maintains; we ship docs + test harness.

9. Connector reference (per-source detail)

Each entry covers: permission model, key APIs, group / nesting behavior, change feed, recommended binding strategy, and known edge cases. The level of detail here is exactly what an engineer needs to start writing the connector.

9.1 Google Drive (including Shared Drives)

Permission model. Five overlapping layers:

- Direct file permissions (user / group / domain / anyone, role of reader / commenter / writer / owner).
- Shared Drive membership: inherited by all files in the drive, with per-file overrides.
- Folder inheritance in My Drive — a file inherits from its parent folder chain.
- Google Groups, recursive (a group can contain groups).
- "Anyone with the link" + "Findable in search" — the trickiest case, because Glean explicitly documents this as the one place they diverge from native Drive search semantics.

Key APIs.

```
# 1. List files + permissions:
files.list(
  pageSize=1000,
  fields='nextPageToken, files(id, name, mimeType, modifiedTime,
    parents, driveId, owners, permissions(id, type,
      emailAddress, domain, role))',
  includeItemsFromAllDrives=True,
  supportsAllDrives=True,
)

# 2. Track incremental changes (the primary mechanism):
token = changes.getStartPageToken().startPageToken
while True:
  resp = changes.list(
    pageToken=token, includeItemsFromAllDrives=True,
    supportsAllDrives=True, includeRemoved=True,
    includePermissionsForView='published',
    fields='nextPageToken, newStartPageToken, changes(...)'
  )
  process(resp.changes); token = resp.newStartPageToken

# 3. Push notifications (avoid constant polling):
changes.watch(
  pageToken=token,
  body={'id': uuid(), 'type': 'web_hook',
    'address': 'https://our.api/webhooks/drive'}
)

# 4. Group expansion via Admin SDK:
groups.list(domain='acme.com')
members.list(groupKey='eng@acme.com') # recursive walk in our code
```

Mapping to unified principals.

```
type=user,    emailAddress=alice@x    →    user:alice@x
```



```

type=group,  emailAddress=eng@x    → group:eng@x
type=domain, domain=acme.com      → domain:acme.com
type=anyone                                     → anyone
Shared Drive D                                → container:drive:D
Folder F in My Drive                      → container:folder:F

```

Auth. Service account with domain-wide delegation. Scopes: `drive.readonly`, `admin.directory.group.readonly`, `admin.directory.group.member.readonly`.

9.2 Confluence Cloud

Permission model. Three layers, increasingly granular: global (who can use Confluence), space (per-space, granted to users or groups), and page restrictions (optional per-page allowlists that override space access).

Key APIs.

```

GET /wiki/api/v2/spaces                # list spaces
GET /wiki/rest/api/space/{key}/permission # space permissions (admin)
GET /wiki/api/v2/pages?space-id=...    # pages in a space
GET /wiki/rest/api/content/{id}/\
    restriction/byOperation/read        # page-level read restriction
GET /wiki/rest/api/group                # groups
GET /wiki/rest/api/group/{name}/member # group members

```

Effective permission for a page.

```

user can read page P IF
  (user OR user's group has space:read on P.space)
  AND (
    P has no read restrictions
    OR user is in P's read restriction allowlist
    OR user's group is in P's read restriction allowlist
  )

```

Change feed. Content webhooks exist; ACL webhooks do not on Cloud. Standard pattern: webhooks for content + nightly full ACL resync. For high-sensitivity tenants, late-bind: at query time, expand `?expand=restrictions` on top-K results before returning.

9.3 Slack

Permission model. Channel-based. Public channels are visible to everyone in the workspace; private channels have an explicit member list; DMs and MPDMs are participants-only.

Key APIs.

```

conversations.list(types='public_channel,private_channel,mpim,im')
conversations.members(channel=C0123)
conversations.history(channel=C0123, oldest=ts)
users.list, usergroups.list, usergroups.users.list

# Events API (real-time):
message                content event
channel_created
member_joined_channel / member_left_channel  ACL events
group_open / group_archive / group_unarchive
team_join              new user event

```

Principal mapping. Each channel is a principal: channel:C0123. User→channel memberships live in the principal expansion service. Public channels carry the special principal workspace:T0XXX. DMs are indexed only when explicitly enabled by the customer (most disable).

9.4 Notion

The hardest of the major SaaS sources for ACLs. Notion's API does not expose per-document permissions — Glean documents this explicitly: anything shared with the integration is visible to all Glean users by default.

- **Strategy.** Treat Notion as a late-binding source. At ingestion, record what the integration is granted access to; at query time, call Notion's API to confirm the requesting user can actually see the page. Cache results for ~60s. Slow, but correct.
- **Alternative.** Some customers can run the Notion enterprise Audit Log + SCIM integration; combined, this gives us per-page access deltas. Not all customers can; we support both.
- **Teamspace.** Notion's teamspace concept maps cleanly to our container principal: container:notion-teamspace:T0123.

9.5 SharePoint / OneDrive

Key APIs.

```
GET /sites/{id}/permissions           # site-level
GET /drives/{id}/items/{item}/permissions # per-item

# Delta query with hierarchical sharing optimization:
GET /drives/{id}/root/delta
  Prefer: hierarchicalsharing
# Without this header, the delta returns sharing for every item
# (inherited or not). With it, sharing is only returned for the
# root of each permission hierarchy + items with explicit changes.

# Azure AD transitive group expansion is native:
GET /groups/{id}/transitiveMembers
```

Sensitivity labels. SharePoint integrates with Microsoft Purview's sensitivity labels (Public / Internal / Confidential / Highly Confidential). We ingest these as PROV-O qualified influence annotations and surface them in our own §11 sensitivity-label layer.

9.6 Jira

- **Two-layer model.** Project permissions (who can browse / create / edit) + issue-level security schemes (what's visible within a project).
- **Issue security levels are the gotcha.** An issue can be restricted to a security level even within an otherwise-open project, and the security level itself has its own principal list.
- **Effective permission:** user can read issue I if (user has BROWSE_PROJECTS on I.project) AND (I.security_level is null OR user is in I.security_level.allowlist).
- **Change feed:** Jira webhooks cover content; permission scheme changes also fire webhooks but are rare; nightly resync is fine.

9.7 Salesforce

The most complex permission model in mainstream SaaS. Five interacting layers. Getting this right is what separates a serious enterprise product from a toy.

- **Organization-Wide Defaults (OWD).** Per-object baseline: Private / Public Read Only / Public Read/Write. Determines what happens when no other rule applies.
- **Role hierarchy.** Vertical inheritance — users higher in the hierarchy see what their subordinates see. Configurable per object via "Grant Access Using Hierarchies".
- **Sharing rules.** Owner-based or criteria-based rules that extend access laterally across the hierarchy. Up to 300 per object.
- **Profiles + permission sets.** Object-level CRUD (Create, Read, Edit, Delete) and field-level access. "View All" / "Modify All" profile permissions override everything below.
- **Manual sharing + Apex-managed sharing.** Record-level shares created ad-hoc or programmatically. The hardest layer to keep in sync — track via Change Data Capture on the `*__Share` tables.

Extraction strategy.

- **Static schema:** read OWDs, profiles, permission sets, role hierarchy, sharing rules via the Metadata API and Tooling API on ingestion and after admin changes.
- **Per-record:** for each indexed record, call `UserRecordAccess` via SOQL to materialize the effective permission. Expensive but correct.
- **Real-time:** subscribe to Salesforce Change Data Capture + Platform Events on Share tables for permission changes; subscribe to standard CDC topics for content changes.
- **Source-recheck fallback:** for high-sensitivity tenants, late-bind by querying `isAccessible` per top-K record at query time. Costly but auditor-proof.

9.8 GitHub (Cloud and Enterprise)

- **Permission layers:** Organization membership, team membership (nested, recursive), repository collaborators, branch protection rules, repository visibility (public / internal / private).
- **Key APIs:** `/orgs/{org}/members`, `/orgs/{org}/teams`, `/teams/{id}/members`, `/repos/{owner}/{repo}/collaborators`.
- **Webhooks cover everything:** team / membership / repository / push / pull_request / branch_protection_rule events. Real-time permission updates are clean.
- **Code search subtleties:** Glean documents that code search needs a separate lifecycle freshness metric — how fast new commits show up in the index — because engineers notice stale indexes within minutes.

9.9 Gmail / Outlook / calendar

- **Permission model:** trivial — each user owns their own mailbox and calendar. Principal mapping is one-to-one with user identity.
- **Caveat:** shared mailboxes, delegate access, and shared calendars exist. Track delegate relationships via Gmail's delegate API / Microsoft Graph delegated permissions.

- **What to NOT do:** never index a user's mailbox under a service-account identity that bypasses per-mailbox ACLs. Always impersonate the user (domain-wide delegation).
- **Calendar:** events have explicit attendee lists; treat the attendee set as the ACL for that event.

9.10 Box / Dropbox

Box exposes per-item collaborations and folder-inherited permissions via well-documented APIs (/collaborations, /folders/{id}/collaborations). Dropbox is similar via the sharing/list_folder_members and sharing/list_file_members endpoints. Both support webhooks for content changes. Both have team-folder concepts that map to our container principal.

9.11 ServiceNow

ITSM and HR data. ACL model is rich: Access Control Records (ACR) evaluated per-row, scripted ACLs, table-level + field-level. The complexity rivals Salesforce. Glean documents that they connect via OAuth with a custom role that gates exactly which tables they index. Recommended: dedicated read-only role, periodic full crawl of permitted tables, ACL extraction via the sys_security_acl table.

9.12 Zendesk / Linear / Asana / Monday / HubSpot

Mid-tier complexity. Each has an organizational permission model (workspace / team / project membership) with REST APIs and webhooks. Standard treatment: poll + webhook combo, nightly ACL resync, team-as-principal mapping.

9.13 Workday

HR system of record. The hardest source for ACL because Workday's security model is its own walled garden — Security Groups, Domain Security Policies, Business Process Security Policies, all with intersection and union semantics. We ingest the org chart (who reports to whom) as a graph signal and treat per-record permissions as a late-binding question to Workday's API. Slow but the only correct approach.

9.14 SQL warehouses (Snowflake, BigQuery, Redshift, Databricks)

This is where Palantir's depth shows. Foundry doesn't just read the data — it reads the row-level and column-level policies and re-implements them in its own enforcement layer.

- **Snowflake:** row access policies + column masking policies, queryable via SHOW POLICIES and DESCRIBE POLICY. ABAC via dynamic data masking with tag-based policies.
- **Databricks:** Unity Catalog with row filters and column masks. Immuta and similar partners add ABAC on top.
- **Redshift:** RLS built on RBAC since 2022, plus column-level controls. Spectrum + Lake Formation for S3-backed tables.
- **BigQuery:** column-level access controls via policy tags, row-level access policies via SQL filters bound to IAM.

- **Microsoft Fabric / OneLake:** OneLake Security in public preview as of 2025; defines table / RLS / CLS / folder roles enforced on the data plane via the authorized-engine model. We register as an authorized engine.

9.15 Generic patterns: S3, HTTP webhook sources, custom

For S3-compatible object stores: standard file-drop modality (§8.5), with sidecar JSON for ACLs. For HTTP webhook sources we don't have a specific connector for: customer wires their source to POST to our generic webhook endpoint, providing event-type and allowed-principals in the payload. For genuinely custom: customer writes a connector using the SDK from §7.

10. Permission model: unified principals

Carried forward from v1 and v2. Every document, fact, and action carries a list of principals who can read (or execute) it. A principal is a typed, prefixed string in a single global namespace:

```

user:alice@acme.com
group:eng-platform@acme.com
domain:acme.com
channel:C0123ABCDEF          (Slack)
space:ENG                    (Confluence)
container:drive:0ABCxyz      (Google Shared Drive)
container:notion-teamspace:T0123
workspace:T0XXX              (Slack workspace)
role:salesforce/SystemAdministrator
label:confidential           (sensitivity label)
anyone                        (public)

```

At query time the requesting user is expanded into their full principal set (themselves, their groups transitively, their domain, their Slack channels, their Confluence spaces, their Salesforce role and its ancestors, etc.). A document is visible if the intersection of its allowed-principal set and the user's expanded set is non-empty AND no deny rule matches.

10.1 Hybrid binding (recap)

Three approaches; we use the hybrid:

- **Early binding:** snapshot ACLs on the chunk at ingestion. Fast but goes stale.
- **Late binding:** look up ACL at query time. Always fresh, slower.
- **Hybrid (default):** denormalize on the chunk for fast filter, SpiceDB live recheck on top-K, optional source recheck on top-N for high-sensitivity tenants. This is what Glean does, and it survives every security review we've benchmarked against.

10.2 SpiceDB schema

Compact, extensible. Covers nested groups, container inheritance, deny rules, and facts-as-ACL-carriers.

```

definition user {}

definition group {
  relation member: user | group#member
  relation manager: user
}

definition domain {
  relation member: user
}

// Containers: Shared Drives, Confluence spaces, Slack channels,
// SharePoint sites, Notion teamspaces, Salesforce roles, ...
definition container {
  relation reader: user | group#member | domain#member
  relation writer: user | group#member
  permission read = reader + writer
}

```

```
}

definition document {
  relation parent: container
  relation reader: user | group#member | domain#member |
    container#read
  relation writer: user | group#member
  relation denied: user | group#member          // explicit deny
  permission read = (reader + writer + parent->read) - denied
}

// Facts carry their own ACL, defaulting to their source episode
definition fact {
  relation source_episode: document
  relation reader: user | group#member
  permission read = reader + source_episode->read
}

// Actions are also gated
definition action_type {
  relation executor: user | group#member
  permission execute = executor
}
```

11. Layered access control

The principal model from §10 governs object-level access (can user U see document D). For enterprise data — especially structured data coming out of warehouses, CRMs, and HR systems — that is not enough. Snowflake, Databricks, OneLake, and Palantir all support additional fine-grained layers, and so must we.

11.1 Five layers

Layer	Granularity	Example	Where enforced
Object	Whole document or record	"Alice can read this contract"	Principal check at retrieval
Row	Single row of a table or record	"Show only opportunities in my region"	Row policy evaluated at query
Column	Field of a record	"Hide SSN from non-HR users"	Column mask applied to output
Cell	Specific field-row combination	"Hide salary only for executives above me"	Cell-level policy at query
Label	Sensitivity / classification tag	"Confidential" — cannot be assembled into context with "Public" without policy approval	Policy engine at context assembly

11.2 Row-level policies

For sources where the row policy is enforced natively (Snowflake, Databricks, BigQuery, Redshift, Fabric / OneLake), we have two choices: replicate the policy in our enforcement layer, or federate the query to the source. We support both, controlled per-source.

```
// Replication mode (default for performance):
// At ingestion, read the source's row policy DDL and translate to
// a SpiceDB-evaluable predicate. Store predicates alongside chunks.
//
// At query time, the retrieval layer evaluates the predicate for
// the requesting user before returning chunks. Failed rows are
// silently filtered.

// Federation mode (default for high-sensitivity):
// At query time, forward the relevant slice of the user's query to
// the source, which enforces its native row policy and returns only
// authorized rows. Slower but auditor-proof.

// Snowflake example (replication):
//   Snowflake DDL:
//   CREATE ROW ACCESS POLICY sales_region AS (region VARCHAR)
//   RETURNS BOOLEAN ->
//   region = CURRENT_REGION() OR IS_ROLE_IN_SESSION('ADMIN')
//   Translated to our predicate language:
//   { type: 'or',
//     clauses: [
```



```
//      { eq: ['$row.region', 'principal.region'] },
//      { in: ['principal.roles', 'role:ADMIN'] }
//    ] }
```

11.3 Column and cell masking

Two roles for column masking in our system:

- **In retrieval results:** when the user queries semantically and a chunk references masked columns, the column values are redacted before being passed to the LLM. The LLM never sees the underlying value.
- **In actions:** when an action's input includes a masked field, the user is prompted with a redacted preview before the action executes.

Cell-level masking is the more general case: the policy depends on both the column and other fields of the row (e.g. "show salary only if employee.manager_id = current_user.id"). Snowflake supports this via row+column policy combinations; we support it via SpiceDB caveats — predicates evaluated against request context.

11.4 Sensitivity labels and policy hooks

Inspired by Microsoft Purview's sensitivity labels and Palantir's dynamic security. Every fact / chunk / action can be tagged with one or more labels (Public / Internal / Confidential / Restricted / PII / PHI / PCI / ITAR / ...). Policy hooks evaluate label combinations at context-assembly time.

```
// Example policy: never assemble PII and Marketing-channel content
// into the same LLM context, even if the user is authorized for both.
policy 'no-pii-in-marketing-context' {
  when:
    context.labels.contains('PII')
    AND context.labels.contains('marketing-channel')
  action:
    drop_chunks where labels.contains('PII')
    log_policy_violation
}
```

This addresses the "oversharing" problem that Knostic and similar tools surfaced for Glean: even perfect ACL enforcement leaves the access-sprawl problem, where users have access to far more than they need, and AI surfaces it efficiently. Sensitivity-label policy is the structural answer.

11.5 Putting it together

The full enforcement pipeline for a query, for clarity:

```
query(user, q):
  // 1. Identity → principal expansion
  principals = expand(user) // user + groups + roles + containers

  // 2. Retrieval with object-level pre-filter
  cand = retrieve(q, filter=principals)

  // 3. SpiceDB authoritative recheck (per chunk)
  cand = batch_check(user, cand)

  // 4. Row-level policy evaluation
```

```
cand = [c for c in cand if row_policy_eval(c, user)]

// 5. Column / cell masking
cand = [apply_masks(c, user) for c in cand]

// 6. Label policy at context assembly
ctx = label_policy_filter(cand)

// 7. Optional source recheck for high-sensitivity tenants
if tenant.high_sensitivity:
    ctx = source_recheck(user, ctx[:N])

// 8. Audit trail (everything is logged)
audit.write(user, q, ctx, principals_hash, ts)

return llm.generate(q, ctx)
```

12. Bitemporal knowledge graph

The structural heart of the platform. The KG is what separates a RAG product from an operational substrate; the bitemporality is what separates an honest operational substrate from a system that lies about state changes.

12.1 Why bitemporal

A flat "current state" KG cannot honestly answer:

- "Who was the account owner when this deal closed?"
- "What was our refund policy on March 3rd, before the audit?"
- "When did we first learn customer X was at risk of churning?"
- "What did Alice know about this incident at 14:00?"

Each requires distinguishing **valid time** (when the fact was true in the world) from **transaction time** (when our system learned it). Zep / Graphiti adopted this for agent memory; Wikidata, YAGO, and NELL all encode some version of it; the BiTRDF model formalizes it for RDF. We adopt it wholesale.

12.2 Fact representation

Every fact is an RDF triple with reified temporal, provenance, and permission metadata:

```
fact:f-c8d1
  rdf:type          :Fact ;
  rdf:subject       acme:Account/123 ;
  rdf:predicate     acme:hasOwner ;
  rdf:object        acme:User/alice@acme.com ;
  :valid_from       "2025-09-01T00:00:00Z"^^xsd:dateTime ;
  :valid_to         "2026-03-15T00:00:00Z"^^xsd:dateTime ;
  :recorded_at      "2025-09-01T00:08:42Z"^^xsd:dateTime ;
  :invalidated_at   "2026-03-14T17:22:01Z"^^xsd:dateTime ;
  :source_episode   doc:salesforce/Opportunity/006xx... ;
  :extracted_by     agent:FactMiner/v3 ;
  :confidence       "0.97"^^xsd:decimal ;
  :allowed_principals (
    "user:alice@acme.com" "group:sales-leadership@acme.com" ).
```

Supersession. Updates never overwrite. A fact whose validity changes is superseded: the old fact's `invalidated_at` is set, and a new fact is appended. Two timelines are always queryable: "what did the world look like at time T" (valid time) and "what did we know at time T" (transaction time).

12.3 The three-subgraph hierarchy (from Graphiti)

- **Episode subgraph.** The raw event log. Every ingested message, document version, ticket update, calendar change, CDC row event. Each carries its original source timestamp. This is the immutable ground-truth corpus.
- **Semantic entity subgraph.** The structured facts extracted from episodes — people, accounts, contracts, projects, and the predicates linking them. Edges carry validity intervals. This is where most queries land.

- **Community subgraph.** Periodically computed clusters (Leiden or Louvain) over the semantic graph, summarized by an LLM. Used for the GraphRAG-style global-summary query: "what were the main themes in customer success this quarter".

12.4 Storage choice

Three viable backend categories. We commit to RDF / SPARQL.

Option	Pros	Cons	Recommendation
Native RDF triple stores (GraphDB, Stardog, Jena Fuseki, Virtuoso)	SHACL native, SPARQL 1.1 native, standards compliance	Smaller ecosystem than property-graph world; performance varies	Primary choice. GraphDB Free for dev, Stardog or Jena Fuseki at scale
Property graphs (Neo4j, AWS Neptune, TigerGraph)	Performance, Cypher familiarity, larger ecosystem	Proprietary semantics; SPARQL via translation if at all; SHACL is bolted on	Considered; not chosen. Possible as a hot-path projection for specific queries
SQL-on-graph (Timbr, PuppyGraph)	Zero-ETL over existing warehouse data; SQL skills reusable	Newer ecosystem; less mature SHACL / temporal story	Watch list. Compelling for customers who want the warehouse to remain the system of record

Why RDF / SPARQL wins for us. Three reasons. (1) Standards compliance is a real differentiator vs. Glean's proprietary EKG and Palantir's closed object model. (2) SHACL is the right shape language for the ontology agents in §14 — executable schema, not a DSL we have to invent. (3) PROV-O integrates natively for §17; with a property graph we'd be reinventing it.

12.5 SPARQL surface

The SPARQL endpoint is exposed three ways:

- **Internal.** The query layer uses SPARQL for KG slices.
- **MCP tool.** The query_sparql tool lets agents write SPARQL directly, with automatic ACL post-filtering (results are stripped of facts the calling user can't read).
- **Natural language.** The assistant translates NL → SPARQL via LLM, then validates with a grammar-constrained decoder (PICARD-style) before execution. This pattern is well-established in recent literature (PIPE-RDF, LLMs4OL).

```
# Example: "who owned this account when the deal closed?"
# valid-time temporal query, generated from NL:
```

```
PREFIX acme: <https://acme.com/ontology#>
PREFIX : <https://platform.internal/schema#>
```

```
SELECT ?owner ?valid_from ?valid_to WHERE {
  ?f rdf:type :Fact ;
    rdf:subject acme:Account/123 ;
    rdf:predicate acme:hasOwner ;
    rdf:object ?owner ;
    :valid_from ?valid_from ;
    :valid_to ?valid_to .
# Deal closed on 2026-02-15:
```

```

FILTER(?valid_from <= "2026-02-15T00:00:00Z"^^xsd:dateTime)
FILTER(?valid_to >= "2026-02-15T00:00:00Z"^^xsd:dateTime)
# Use the fact as we knew it as of now (transaction time):
FILTER(NOT EXISTS { ?f :invalidated_at ?inv })
}

```

13. Ontology layer (OWL + SHACL)

Two layers: a stable upper ontology that we ship, and a customer ontology that evolves with the business. Both are OWL files, both are accompanied by SHACL shapes, both are versioned in Git so all changes are reviewable as PRs.

13.1 Upper ontology

Concepts every enterprise has, with stable IRIs we never break. Includes the temporal and provenance vocabulary that everything else builds on.

- **Foundational:** Person, Organization, Document, Conversation, Message, CalendarEvent, EmailMessage, Project, Issue, Repository, Commit, FileObject.
- **Temporal:** the BiTRDF vocabulary — `valid_from`, `valid_to`, `recorded_at`, `invalidated_at`, and a BiTRDF-style `tref` resource that represents a temporal reference.
- **Provenance:** W3C PROV-O terms — `prov:Entity`, `prov:Activity`, `prov:Agent`, `prov:wasGeneratedBy`, `prov:wasDerivedFrom`, `prov:wasAttributedTo`. We reuse the published ontology rather than re-inventing.
- **Permission:** our principal namespace types — `Principal`, `UserPrincipal`, `GroupPrincipal`, `ContainerPrincipal`, `SensitivityLabel`.
- **Actions:** `ActionType`, `ActionInvocation`, `ActionParameter`, linking objects (subjects of actions) to functions (the executable logic).

13.2 Customer ontology

Domain-specific. An investment firm has `Fund`, `LimitedPartner`, `CapitalCall`, `Distribution`. A SaaS company has `Account`, `OpportunityStage`, `Renewal`, `ExpansionEvent`. A hospital has `Patient`, `Encounter`, `LabResult`. These are proposed by the ontology agents (§14), refined by humans, and committed to the customer's OWL repository.

```

# customer/acme/ontology.ttl
@prefix owl: <http://www.w3.org/2002/07/owl#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .
@prefix upper: <https://platform.internal/upper#> .
@prefix acme: <https://acme.com/ontology#> .

acme:Account a owl:Class ;
  rdfs:subClassOf upper:Organization ;
  rdfs:label "Account"@en ;
  rdfs:comment "A customer organization in our CRM."@en .

acme:hasOwner a owl:ObjectProperty ;
  rdfs:domain acme:Account ;

```

```

rdfs:range upper:Person ;
rdfs:label "has owner"@en .

acme:tier a owl:DatatypeProperty ;
  rdfs:domain acme:Account ;
  rdfs:range xsd:string .

```

13.3 SHACL shapes as the contract

Every customer class has SHACL shapes. Fact-mining agents (§15) produce facts only if they conform. Ontology agents (§14) propose shape changes when they see persistent violations. SHACL is executable schema, not a docstring.

```

@prefix sh: <http://www.w3.org/ns/shacl#> .
@prefix acme: <https://acme.com/ontology#> .

acme:AccountShape a sh:NodeShape ;
  sh:targetClass acme:Account ;
  sh:property [
    sh:path acme:hasName ;
    sh:datatype xsd:string ;
    sh:minCount 1 ;
    sh:maxCount 1 ;
  ] ;
  sh:property [
    sh:path acme:hasOwner ;
    sh:class upper:Person ;
    sh:minCount 1 ;
  ] ;
  sh:property [
    sh:path acme:tier ;
    sh:in ( "enterprise" "midmarket" "smb" ) ;
  ] .

```

Initial shapes are generated automatically from the OWL ontology using Astrea or OWL2SHACL. Both are well-established tools in the semantic-web community. The ontology agents then refine them based on what they observe in the actual data.

14. Ontology agents

An ontology is dead the day it's written. Schemas evolve as the business does — new product lines, new lifecycle stages, new contract types. Ontology agents are autonomous workers that keep the customer ontology and its SHACL shapes in sync with reality, by continuously mining the actual data.

14.1 Three sub-roles

- **Class proposer.** Watches incoming episodes. When it sees a cluster of entities that don't fit any existing class but recur (e.g. "capital call" mentioned 40 times this month in finance documents), it proposes a new class with a candidate OWL definition and example instances.
- **Property refiner.** Watches facts being rejected by SHACL. If 30% of rejected facts have a specific property value not in the `sh:in` enumeration, it proposes a shape relaxation. Conversely, if 99.9% of facts in a property have integer values but the shape allows strings, it proposes tightening.

- **Ontology aligner.** Detects when two customer-defined classes are likely the same (e.g. "Customer" and "Account" used with the same property fingerprint) and proposes a merge or owl:equivalentClass alignment.

14.2 Workflow

```
OntologyAgent.run():
    episodes = bitemporal_kg.episodes_since(last_run)
    candidates = LLM.extract_class_candidates(episodes,
                                              current_ontology)

    for c in candidates:
        if frequency(c) > threshold and not_subsumed_by(c,
                                                         current_ontology):
            shapes = generate_shacl(c, observed_instances)
            proposal = OntologyProposal(
                class=c,
                parent=infer_parent(c, current_ontology),
                shapes=shapes,
                supporting_episodes=top_k_episodes(c),
                confidence=score(c))
            review_queue.put(proposal)

# Recent literature (LLMs4OL Challenge 2025, OntoEKG 2026)
# shows multi-agent decomposition outperforms single-agent
# extraction; we use one agent per sub-role + a coordinator.
# F1 ≈ 0.72 on the OntoEKG benchmark; we target 0.85+ on
# customer-specific data via in-context examples from the
# customer's own approved ontology.
```

14.3 Human-in-the-loop, Palantir-style branching

Ontology changes are governance events. They are never auto-applied. Palantir uses Branching and Proposals — we adopt the same model:

- Every proposal opens a Git PR against the customer's ontology repo.
- The PR includes the OWL diff, the SHACL diff, supporting episodes (2-10 examples), and a confidence score.
- Reviewers approve or reject; on approve, the change cascades — shapes recompile, fact-mining agents re-run on recent episodes under the new shapes, existing facts are revalidated.
- Branching means changes can be proposed and tested in isolation before merging to the main ontology — same pattern that engineering teams already understand from code.

15. Fact-mining agents

Where ontology agents shape the schema, fact-mining agents fill it. They run continuously, scanning new documents, conversations, calendar events, and CRM updates, and extracting structured facts that conform to the current SHACL shapes.

15.1 Per-episode pipeline

```
FactMiningAgent.process(episode):
    # 1. ACL check – only mine from data the agent can see
    if not authorized(agent.principal,
                      episode.allowed_principals):
        return # skip

    # 2. Entity extraction with the customer's ontology
    entities = LLM.extract_entities(
        episode, classes=ontology.classes,
        examples=ontology.examples)

    # 3. Entity resolution (see §16)
    entities = entity_resolver.resolve(entities)

    # 4. Relation extraction
    triples = LLM.extract_triples(
        episode, properties=ontology.properties,
        entities=entities)

    # 5. Temporal extraction (the tricky one)
    for t in triples:
        t.valid_from, t.valid_to = LLM.extract_validity(t, episode)
        # "Alice was promoted to VP last Tuesday" → resolve
        # "last Tuesday" relative to episode.timestamp

    # 6. SHACL validation
    result = shacl_engine.validate(triples, current_shapes)
    rejected = result.failures
    if rejected:
        feedback_loop.send(rejected) # informs property refiner

    # 7. Supersession
    for t in result.accepted:
        existing = kg.find_overlapping(t.subject, t.predicate,
                                      t.valid_from)
        if existing and conflicts(existing, t):
            kg.supersede(existing, t)
        else:
            kg.insert(t)

    # 8. Provenance: every triple carries source_episode,
    #     extracted_by (this agent + version), confidence
    # 9. ACL: inherit allowed_principals from episode unless
    #     ontology pins it (some classes have static ACLs)
```

15.2 Confidence and human review thresholds

Every fact carries a confidence score. Facts below a configurable threshold (default 0.7) go to a review queue instead of the live KG. The threshold is per-class:

```
# customer/acme/extraction_policy.yaml
thresholds:
  acme:Contract.hasParty:      0.95 # contract parties matter
  acme:Contract.hasEffectiveDate: 0.90
  acme:Account.hasOwner:      0.85
```



```

acme:Meeting.hasAttendee:      0.50  # easy and low-stakes
upper:Person.hasEmail:         0.70  # default

auto_approve_below: 0.0        # never auto-approve
auto_reject_below:  0.30       # always reject

```

15.3 Agents that mine company data

The platform ships several pre-built mining agents. Customers can add more via the SDK.

Agent	Source episodes	Facts produced
ConversationMiner	Slack, Teams, Zoom transcripts, email	Decisions, action items, attendees, sentiment, topic clusters
TicketMiner	Jira, Linear, ServiceNow, Zendesk	Issue lifecycle, ownership transitions, blockers, resolution patterns
CrmMiner	Salesforce, HubSpot CDC	Account changes, opportunity stage transitions, renewal signals, churn predictors
DocumentMiner	Drive, Confluence, Notion, SharePoint	Document-to-project links, authorship, decisions, deadlines
CodeMiner	GitHub, GitLab, Bitbucket	Service ownership, dependency edges, deployment events, incident-to-commit links
CalendarMiner	Google Calendar, Outlook	Meeting topics, attendee networks, recurring patterns, schedule changes
FinanceMiner	ERP CDC, expense systems	Spend by vendor, budget tracking, anomaly signals
HrMiner	Workday, BambooHR	Org structure, manager changes, employment events

15.4 Agents that just retrieve facts for their own decisions

A primary use case: agents that don't insert anything, they only retrieve. Our MCP server exposes the read surface; any agent running anywhere can connect.

Examples we expect to see in week one:

- **On-call agent.** Paged at 3am. Calls `list_facts_about(service:checkout-api)` and `search("recent incidents checkout")`. Drafts a runbook delta. Never opens our UI.
- **Sales agent in Cursor.** Engineer is writing a customer-specific integration. Cursor (via our MCP server) calls `get_fact(subject=acme:Account/123, predicate=acme:integrationConfig)`. Provides correct config inline.
- **Renewals agent.** Cron job. SPARQL query for accounts whose `acme:nextRenewalDate` is within 60 days AND whose `acme:healthScore` dropped > 15 points in the last 30 days. Posts to Slack with summary.
- **Compliance agent.** Runs at month-end. SPARQL for all decisions in `acme:HighRiskCategory` made without a written approval fact in the KG. Drafts the audit package.

The platform is the substrate; the agents are the applications. This is the Palantir AIP positioning, except agents can be built by anyone, hosted anywhere, and authenticated via standard OAuth.

16. Entity resolution

Without good entity resolution, the KG splinters. "Alice", "Alice Smith", "alice@acme.com", "Alice Smith (sales)", and u_8a9d2e all need to collapse to one node. Gartner estimates poor data quality (largely duplicates) costs the average enterprise \$13M annually. Children's Medical Center Dallas reduced patient duplicates from 22% to 0.14% with proper entity resolution. For an enterprise KG, this is infrastructure, not a feature.

16.1 Cascade matching

Three tiers, decreasing in volume, increasing in cost per call:

```
def resolve(rec_a, rec_b):
    # Tier 1: rules — ~40% of decisions, free
    if exact_id_match(rec_a, rec_b):
        return MATCH, 1.0
    if definite_non_match(rec_a, rec_b): # different orgs, etc.
        return NO_MATCH, 1.0

    # Tier 2: ML — ~55% of decisions, cheap
    ml_score = ditto_or_finetuned_model.predict(rec_a, rec_b)
    if ml_score > 0.9:
        return MATCH, ml_score
    if ml_score < 0.3:
        return NO_MATCH, 1 - ml_score

    # Tier 3: LLM — ~5% of decisions, edge cases only
    return llm_match(rec_a, rec_b)
```

16.2 Semantic blocking

Pairwise comparison is $O(n^2)$ and impractical at enterprise scale. We use embedding-based blocking: cluster candidate records by embedding similarity (top-50 nearest neighbors), then run the cascade only within each cluster. This brings the effective comparison count from n^2 to roughly $50n$. MERAI and similar production systems scale to 15M+ records this way.

16.3 The transitive-closure trap

Matching produces pairs; we need clusters. Naive transitive closure ($A=B, B=C \Rightarrow A=B=C$) is dangerous: a few false-positive pairs in a dense graph collapse everything into one giant supercluster. We use connected components with a confidence threshold edge: only edges above 0.85 contribute to closure, and we cap cluster size with a sanity check ("is it plausible that this person appears 47 times in our data?").

17. Provenance, lineage, audit (W3C PROV)

Every fact in the KG and every action executed by the action runner generates a PROV-O entry. PROV is the W3C recommendation for provenance; we use it because it's standard, machine-readable, and survives audits.

17.1 The PROV trinity

- **Entity.** A fact in the KG, a document, a chunk, an action result.
- **Activity.** An extraction run, an action execution, an ontology approval.
- **Agent.** A user, a service account, a fact-mining agent, an ontology agent.

Standard relations:

```

prov:wasGeneratedBy    : entity ← activity
prov:wasDerivedFrom    : entity ← entity
prov:wasAttributedTo   : entity ← agent
prov:wasAssociatedWith : activity ← agent
prov:used               : activity ← entity
prov:startedAtTime     : activity → xsd:dateTime
prov:endedAtTime       : activity → xsd:dateTime

```

17.2 Bi-temporal audit log

All permission-service changes are appended to a bi-temporal audit log, modeled the same way as facts in the KG. "Who had access to X at time Y" is a single SPARQL query.

```

PREFIX prov: <http://www.w3.org/ns/prov#>
PREFIX : <https://platform.internal/schema#>

# "Who had access to document D on 2026-02-15?"
SELECT ?user WHERE {
  ?perm a :PermissionGrant ;
        :resource doc:D ;
        :grantee ?user ;
        :valid_from ?vf ;
        :valid_to ?vt .
  FILTER(?vf <= "2026-02-15T00:00:00Z"^^xsd:dateTime)
  FILTER(?vt >= "2026-02-15T00:00:00Z"^^xsd:dateTime)
}

```

17.3 OpenLineage integration

For the ingestion pipeline, we also emit OpenLineage events. This lets customers see our pipeline alongside their dbt, Airflow, and Spark jobs in their existing lineage tools. OpenLineage is the operational counterpart to W3C PROV: PROV for the KG facts, OpenLineage for the pipeline.

18. Retrieval pipeline (hybrid)

Hybrid retrieval over three substrates: the vector index, the bitemporal KG (via SPARQL), and BM25 lexical search. The right substrate depends on the question:

- **Semantic similarity** ("find me docs about Q3 strategy") → vector.
- **Structured** ("who owns account 123 as of last March") → SPARQL with valid_at.
- **Lexical** ("find the contract with clause 4.2.1") → BM25.
- **Multi-hop** ("all renewals at risk because their CSM left") → SPARQL + vector hybrid.

18.1 Reference flow

```
def query(user, q):
    principals = principal_cache.load(user)
    intent      = classify_intent(q)

    candidates = []
    if intent.uses_vector:
        candidates += vectordb.search(
            embed(q), k=50,
            filter=allowed_principals_intersects(principals))

    if intent.uses_kg:
        sparql = nl_to_sparql(q, ontology)
        sparql = grammar_constrained_repair(sparql)
        candidates += kg.execute(sparql, principals=principals)

    if intent.uses_lexical:
        candidates += bm25.search(
            q, k=50,
            filter=allowed_principals_intersects(principals))

    # Reciprocal Rank Fusion across substrates
    fused = rrf(candidates)
    # Cross-encoder for top-50 only (expensive)
    reranked = cross_encoder.rerank(q, fused[:50])

    # Authoritative recheck (see §10, §11)
    approved = batch_acl_check(user, reranked)
    if tenant.high_sensitivity:
        approved = source_recheck(user, approved[:10])

    audit.write(user, q, approved, ts=now(),
                principals_hash=hash(principals))
    return llm.generate(q, ctx=approved[:10])
```

18.2 Vector index choice

Three viable choices in 2026. We default to pgvector for development and small-to-mid customers, and offer Qdrant or Turbopuffer at scale.

Choice	When
pgvector 0.9+	Default. Up to ~10M vectors, customer wants one fewer system to operate, ACID consistency with relational metadata. The industry's actual default in 2026.
Qdrant	Self-hosted preference, Rust performance, rich JSON-payload filtering for our metadata-heavy use case. Strong for the 10M-100M range.
Turbopuffer	S3-backed, no namespace limit, hybrid search built in even on low tiers. Best for multi-tenant SaaS where tenant isolation via namespaces matters.
Weaviate	If we lean heavier on built-in BM25 + vector hybrid. Considered.
Vespa	Billion-scale hybrid retrieval. Future option, overkill today.

18.3 Graph-distance reranker (from Zep)

For KG-heavy queries, results closer to a designated centroid entity get a boost. The centroid is usually the user themselves (personalization) or the entity the query is about. This is the Zep-paper episode-mentions reranker, applied to the customer KG.

19. Actions: the kinetic layer

Palantir's biggest architectural innovation, and what separates an operational platform from a search product. Objects are nouns; actions are verbs. A read-only knowledge graph is interesting; a knowledge graph paired with governed, audited, write-back actions is a substrate for AI agents that actually do work.

19.1 Action types

An action type is a declarative description of an operation: its input schema (JSON Schema), the principals allowed to execute it, the validation logic, the side effects, and the audit requirements. Every connector that supports writes contributes action types to the registry.

```
# Example action type declaration
ActionType:
  id: salesforce.update_opportunity_stage
  source: connector:salesforce
  description: |
    Move a Salesforce Opportunity to a new stage. Records a fact
    in the KG capturing who triggered, when, and prior state.
  input_schema:
    type: object
    properties:
      opportunity_id: { type: string }
      new_stage:
        type: string
        enum:
          ["Prospecting", "Qualification", "Proposal",
           "Negotiation", "Closed Won", "Closed Lost"]
      reason: { type: string, maxLength: 500 }
    required: [opportunity_id, new_stage]

  permission:
    required_principals:
      - role:salesforce/SalesUser
      OR
      - group:revops@acme.com

  validation:
    # Custom function (TypeScript / Python / WASM)
    - fn:validate_stage_transition
    - fn:require_close_reason_when_closed_lost

  approval:
    # Optional approval workflow for high-value opps
    when: "opportunity.amount > 500000"
    approvers: [role:salesforce/SalesManager]

  side_effects:
    - kind: source_writeback
      connector: salesforce
      operation: PATCH /subjects/Opportunity/{opportunity_id}
    - kind: kg_fact_emit
      class: acme:OpportunityStageTransition
    - kind: audit_log
    - kind: notify
      channel: slack:#revops
```

19.2 Idempotency and exactly-once semantics

Every action carries an idempotency key (client-provided or server-generated). Retried calls with the same key return the original result rather than executing twice. For source writes, this means we never accidentally create duplicate Jira tickets or update the same Salesforce record twice. Inspired by Stripe's API and Palantir's atomic-transaction model.

19.3 Approval workflows

High-stakes actions (financial, legal, anything tagged `label:restricted`) can require approval before execution. The approval queue is the same infrastructure as ontology proposals from §14 — proposal-and-review, with full provenance, comment threads, and PR-style approvals.

20. MCP integration

20.1 Outbound: the platform as an MCP server

The platform ships an MCP server. Any MCP-aware client connects with a single OAuth flow and gets the platform's capabilities as native tools. Not doing this in 2026 means being invisible to where developers and analysts actually live.

Tool surface (initial):

```
search(query, k=10, source_filter=None, valid_at=None)
    Hybrid retrieval; respects calling user's ACL automatically.

get_fact(subject, predicate=None, valid_at=None, as_of=None)
    Direct fact lookup, with bitemporal context.

query_sparql(query, valid_at=None, as_of=None)
    Raw SPARQL with automatic ACL post-filter.

list_facts_about(entity_iri, since=None)
    All facts where entity is subject or object.

time_travel_query(query, valid_at)
    Answer "as of date X".

propose_fact(subject, predicate, object,
             valid_from, valid_to, evidence)
    Write a fact (SHACL + ACL + review-queue gated).

execute_action(action_type, input, idempotency_key)
    Run a kinetic action; the user-facing trigger from agents.

list_proposals(status="pending")
approve_proposal(proposal_id, comment)
    Ontology evolution surface.

get_provenance(fact_id)
    Episodes, agent, confidence, timestamps.
```

20.2 Authentication

OAuth 2.1 with PKCE. The MCP client receives a bearer token bound to the end-user's enterprise identity. All tool calls execute with that identity — ACL filtering is automatic. No service-account shortcuts, no shared keys. Tokens are short-lived (1 hour) with refresh. This is the same pattern Supabase and other production MCP servers have converged on.

20.3 Cursor and Claude Code specifics

Both support stdio and HTTP/SSE MCP transports. We ship:

- **Remote HTTP/SSE server.** Primary deployment. Single endpoint, OAuth flow, scoped per workspace.
- **"Add to Cursor" / "Add to Claude Code" buttons.** One-click install from our documentation page.
- **Project-scoped config.** Docs show how to add the server to `.cursor/mcp.json` or Claude Code's MCP config for team-shared setup.
- **Local stdio shim.** A Node CLI wraps the remote API for offline / on-prem deployments. Useful for security-paranoid customers.

```
// .cursor/mcp.json (committed to the customer's repo)
{
  "mcpServers": {
    "acme-knowledge": {
      "type": "http",
      "url": "https://mcp.platform.example.com/v1",
      "auth": { "type": "oauth" }
    }
  }
}
```

20.4 Inbound: the platform as an MCP host

Where a source system itself ships an MCP server (Supabase, GitHub, Linear, Airbyte, etc.), our connector layer can consume it directly instead of writing a bespoke connector. We still need to extract ACLs separately — most source MCP servers don't expose permission data — but it shortens time-to-integration for the content channel.

21. Reference deployment topology

Four supported topologies. Customers choose based on data sensitivity, regulatory requirements, and operational appetite.

21.1 Topology A: Multi-tenant SaaS (default)

We host. Tenants are isolated by `tenant_id` prefix in all storage. Connectors run in our cloud, source credentials stored in our KMS, all data in our region of choice (US, EU, AU at GA).

- **Pros:** fastest onboarding (minutes), lowest TCO for the customer, fastest iteration on our side.
- **Cons:** some regulated industries (defense, certain HIPAA customers, EU sovereign cloud requirements) cannot use shared infrastructure.
- **Tenant isolation:** separate SpiceDB schemas per tenant, separate Kafka topics per tenant, separate RDF graphs per tenant. Vector index uses tenant-id as a hard filter at the engine level.

21.2 Topology B: Customer VPC (BYOC)

We deploy into the customer's cloud account (AWS, Azure, GCP) with our Terraform / Helm modules. The control plane stays in our cloud; the data plane (connectors, storage, query layer, agents) runs in the customer's VPC. Inspired by Airbyte Self-Managed Enterprise's BYOC option.

- **Pros:** data never leaves the customer's network. Satisfies most enterprise security reviews.
- **Cons:** slower onboarding (days, not minutes), heavier support burden. Customers prefer this — we should plan to support it from M5.

21.3 Topology C: Single-tenant managed SaaS

We host, but in a dedicated cluster per customer. Same code paths as multi-tenant SaaS, different operational profile. Useful for large customers who want SaaS economics but need physical isolation.

21.4 Topology D: Fully self-hosted (open core)

The core platform is open source under Apache 2.0. Customers run the whole thing on their own infrastructure, with no calls home. Commercial features (managed connectors, advanced agents, compliance dashboards, support) require a license but are optional. This is Onyx's model and it's the only way to win the most security-conscious customers — and to be honest about "available to everyone".

22. Security model

22.1 Threat model

- **Untrusted query user.** Cannot see content they couldn't see in source.
- **Compromised user credentials.** Blast radius limited to what that user could access. No privilege escalation through us.

- **Malicious tenant admin.** Can grant themselves access in our system, but the source recheck for high-sensitivity tenants prevents bypassing source ACLs.
- **Compromised connector credentials.** Rotated regularly, scoped narrowly, stored in HSM-backed KMS. Per-tenant credentials never reused.
- **Prompt injection from indexed content.** Mitigated by treating all indexed content as untrusted, requiring citations for all generated claims, and never auto-executing tool calls that originated from indexed-content suggestions.
- **Insider threat at us.** Customer-managed encryption keys (CMEK) so even our SRE cannot decrypt customer data. Audit log of every access by our staff. SOC 2 Type 2 + ISO 27001 from M9.

22.2 Defenses by failure mode

Failure mode	Defense
Stale group membership	Group-change events invalidate principal cache directly; 60s TTL backstop.
Stale document ACL	SpiceDB recheck on top-K; source recheck for high-sensitivity tenants on top-N.
Information leakage via result count	Always backfill from oversampled candidates; never return short result sets silently.
Leakage via LLM synthesis	ACL filter at context assembly, before any model call. No post-hoc redaction.
New-enemy problem	SpiceDB ZedTokens for read-after-write consistency during sensitive sync windows.
Prompt injection	Untrusted-content treatment; citation requirement; tool-call quarantine for indexed-content suggestions.
Oversharing	Sensitivity-label policy (§11.4), access-pattern analytics for admins.
Tenant-isolation breach	Tenant ID as a hard filter at every storage layer; per-tenant encryption keys; quarterly red-team exercises.

23. Performance, scale, SLAs

23.1 Initial scale targets

Documents	50M (per tenant)
Facts	100M (per tenant)
Permission relationships	10M (per tenant)
Users	100K (per tenant)
Groups	10K (per tenant)
Sources per tenant	20-30 at GA
Concurrent users	5K (per tenant)
Sustained QPS	100 (per tenant)
Burst QPS	1K (per tenant)

23.2 Latency budget

Stage	P50	P95	P99
Principal expansion (cached)	5ms	15ms	40ms
Vector retrieval (k=50)	30ms	80ms	180ms
SPARQL retrieval (typical)	40ms	120ms	300ms
BM25 retrieval	20ms	50ms	100ms
RRF fusion + reranker	60ms	150ms	300ms
SpiceDB batch_check (50 items)	10ms	30ms	80ms
Source recheck (top-3, optional)	200ms	500ms	1500ms
LLM generation (excluded from us)	—	—	—
End-to-end (no source recheck)	200ms	500ms	1100ms

23.3 Freshness SLAs

- **Webhook-backed sources:** $P95 \leq 30$ seconds from source event to indexed.
- **CDC-backed sources:** $P95 \leq 60$ seconds.
- **Poll-backed sources:** bounded by configured poll interval (typically 5-15 minutes).
- **ACL changes:** $P95 \leq 5$ minutes; hard ceiling 1 hour, backstopped by source recheck for high-sensitivity tenants.
- **Ontology approval-to-effect:** ≤ 5 minutes (shape recompile + validator restart).

24. Build vs. buy decisions

Where we write code, where we adopt open source, where we license. Critical for keeping the engineering org small without compromising the platform.

Component	Decision	Rationale
Connector framework	Build	Our unique value. SDK contract + permission model.
Managed connectors (top 10)	Build	Quality and ACL fidelity are the core moat.
Long-tail connectors	Adopt OSS (Airbyte-format compatible)	Reuse 600+ Airbyte / Singer connectors for content; pair with custom ACL extraction.
CDC infrastructure	Adopt Debezium	Industry standard, mature, supports every major DB.
Event bus	Adopt (Kafka or Redpanda)	Not our differentiator.
Authorization engine	Adopt SpiceDB	Most mature open-source Zanzibar implementation.
Vector DB	Adopt (pgvector default, Qdrant at scale)	Commodity; pick best of breed.
RDF triple store	Adopt (GraphDB Free dev, Stardog or Jena Fuseki prod)	Standards compliance matters more than implementation.
SHACL validator	Adopt (TopBraid SHACL, pySHACL)	Mature OSS.
Ontology IDE for customers	Adopt (Protégé compatibility) + build a web UI on top	Web UI for casual users; power users use Protégé.
Entity resolution	Build (cascade) + adopt base models	ML core can use OSS (Ditto, DeepMatcher); cascade and blocking are ours.
LLM(s) for extraction	Adopt (multi-provider abstraction)	Don't pick one; ride the cost / quality curve. Anthropic, OpenAI, Google, Mistral, plus local Qwen 3 / Llama 4 for private deploys.
MCP server	Build	The shape of our tool surface is our product.
Audit log + lineage	Build (PROV-O) + adopt (OpenLineage)	PROV in the graph; OpenLineage for the pipeline.
Identity provider integration	Adopt (OAuth 2.1 + OIDC + SCIM via standard libs)	Commodity.
Observability	Adopt (OpenTelemetry, Prometheus, Grafana)	Standard.

25. Milestones

Twelve phases, each with a clear shippable deliverable. The first six are foundation; the next six are product expansion.

Phase	Deliverable	Notes
M1	Permission service + first connector	SpiceDB schema, materialized ACL, principal expansion cache. Google Drive end-to-end with changes.list.
M2	Vector retrieval with ACL filter	pgvector, oversampled retrieval, SpiceDB recheck, backfill loop, audit log. Parity with v1 scope.
M3	Bitemporal KG core	GraphDB committed for dev. Ingestion writes facts with valid + transaction time. Internal SPARQL endpoint.
M4	Upper ontology + SHACL validation	Ship Person / Document / Conversation / Project / Issue / Email vocab. SHACL on insert. Astrea-generated initial shapes.
M5	Connector breadth — top 8	Confluence, Slack, Notion, SharePoint, Jira, Salesforce, GitHub, Gmail. Common SDK proven in production.
M6	MCP server v1	search, get_fact, query_sparql, time_travel_query. OAuth, remote HTTP/SSE. Cursor and Claude Code documented.
M7	Fact-mining agents v1	ConversationMiner, DocumentMiner, CrmMiner. Confidence scoring. Review queue.
M8	Ontology agents v1	Class proposer + property refiner. Review UI. Per-customer OWL in Git.
M9	Hybrid retrieval + community subgraph	Vector + SPARQL + BM25 fusion. Community detection + summaries. Graph-distance reranker.
M10	Layered access control	Row + column + cell policies. Sensitivity labels. Policy hooks at context assembly. Source recheck mode generally available.
M11	Action runner / kinetic layer	Write-back actions. Idempotency. Approval workflows. Initial action registry for top 5 sources.
M12	Self-hosted distribution + open-source release	Apache 2.0 core. Helm charts. Terraform modules. Quickstart for BYOC and fully-self-hosted deployments.

26. Open questions

- **RDF store at scale.** GraphDB Free for dev is settled. Stardog vs. Jena Fuseki vs. self-hosted Virtuoso for production needs a spike under realistic load (100M triples, 100 QPS).
- **Property-graph projection.** If SPARQL performance becomes problematic for specific hot queries, do we maintain a Neo4j projection? Cost: operational complexity. Benefit: 10x latency on hot paths.
- **MCP server pricing.** Free for individuals, metered for enterprise, or bundled? Affects M6 scope and our positioning vs. Glean, Dust.
- **SPARQL exposure to customers.** Power users will want it; security-conscious customers may not. Default off, opt-in per tenant?

- **Ontology-agent autonomy.** Which classes of changes can auto-apply (e.g. shape relaxations below threshold), vs. always human-reviewed?
- **Fact-mining budget.** Each customer has a finite token budget for ongoing extraction. Default and metering?
- **Multi-tenancy isolation.** Separate SpiceDB instances per tenant or shared with strict prefixes? Glean isolates; we should probably too.
- **Distillation.** Can we fine-tune small models (Qwen 3, Llama 4) on customer-specific extraction once we have enough labeled data? Reduces extraction cost by 10x at the price of model-management complexity.
- **Bring-your-own-LLM.** Customers want to choose. Glean ships a 15-LLM hub; what's the right v1 set for us? Anthropic, OpenAI, Google, plus one local? Plus Mistral for EU customers?
- **Action registry.** Curated vs. customer-extensible? Probably both — curated for connector-provided actions, customer-extensible via SDK.

27. References

Foundational papers and standards

- Pang et al., *Zanzibar: Google's Consistent, Global Authorization System*, USENIX ATC 2019.
- Rasmussen et al., *Zep: A Temporal Knowledge Graph Architecture for Agent Memory*, 2025 (arXiv:2501.13956).
- Microsoft Research, *From Local to Global: A Graph RAG Approach to Query-Focused Summarization*, 2024.
- Oyewale, *LLM-Driven Ontology Construction for Enterprise Knowledge Graphs (OntoEKG)*, Liber AI Research, 2026.
- Cimmino et al., *Astrea: Automatic Generation of SHACL Shapes from Ontologies*, 2020.
- *On the Implementations of the BiTemporal RDF Model*, Informatics, 2026.
- Anselma et al., *Bitemporal Property Graphs*, ADBIS 2026.
- Ranganath, *PIPE-RDF: LLM-Assisted Pipeline for Enterprise RDF Benchmarking*, UC San Diego.
- *HKUDS LightRAG: Simple and Fast Retrieval-Augmented Generation*, EMNLP 2025.
- Belmabrouk et al., *A Multi-LLM Agent System for Modular Ontology Population*, 2026.
- *MERA1: Robust and Efficient Pipeline for Enterprise-Level Large-Scale Entity Resolution*, 2025.
- W3C: RDF 1.1, OWL 2, SHACL, SPARQL 1.1, PROV-O.
- *Model Context Protocol Specification*, modelcontextprotocol.io.
- *OpenLineage Specification*, LF AI & Data Foundation.

Product references (architectural inspiration)

- Palantir Foundry: Ontology Architecture docs; "The Ontology System" (Architecture Center); "Connecting AI to Decisions with the Palantir Ontology" (Palantir Blog, Nov 2025).

- Glean: Connectors documentation; Permissions and ACLs guide; Indexing API documentation; Futurum Group (April 2026) on Glean's \$200M ARR; TechCrunch (Feb 2026) "The enterprise AI land grab".
- Microsoft Fabric / OneLake: Security overview; OneLake Security public preview docs; Microsoft Purview integration.
- Snowflake / Databricks / Redshift / BigQuery: row-level and column-level access control documentation.
- Airbyte, Nango, Fivetran, Hotglue: connector taxonomy and tiering models.
- Debezium: CDC architecture and connector library.
- SpiceDB (Authzed) and OpenFGA: Zanzibar-derived authorization engines; ZedTokens for read-after-write consistency.
- Pinecone, Qdrant, Weaviate, Turbopuffer, LanceDB: 2026 vector database comparisons.
- Cursor and Claude Code MCP documentation; Supabase MCP server as a reference for production MCP architecture.

Inspiration we've earned the right to cite

Palantir for the ontology shape. Glean for ACL discipline and the MCP server/host duality. Zep / Graphiti for bitemporality. Microsoft Fabric for the authorized-engine model. Foundry's branch-and-proposal pattern for ontology evolution. Stripe for idempotency. Stripe for documentation, frankly.

This is a living document. Comment threads open in Notion. Reviewable diffs in Git:

platform/rfcs/001-permission-aware-enterprise-kg.md. Substantive changes require maintainer review; typos and formatting may be merged on green CI.

13. Ontology layer (OWL + SHACL)

Two layers: a stable upper ontology that we ship, and a customer ontology that evolves with the business. Both are OWL files, both are accompanied by SHACL shapes, both are versioned in Git so all changes are reviewable as PRs.

13.1 Upper ontology

Concepts every enterprise has, with stable IRIs we never break. Includes the temporal and provenance vocabulary that everything else builds on.

- **Foundational:** Person, Organization, Document, Conversation, Message, CalendarEvent, EmailMessage, Project, Issue, Repository, Commit, FileObject.
- **Temporal:** the BiTRDF vocabulary — `valid_from`, `valid_to`, `recorded_at`, `invalidated_at`, and a BiTRDF-style `tref` resource that represents a temporal reference.
- **Provenance:** W3C PROV-O terms — `prov:Entity`, `prov:Activity`, `prov:Agent`, `prov:wasGeneratedBy`, `prov:wasDerivedFrom`, `prov:wasAttributedTo`. We reuse the published ontology rather than re-inventing.

- **Permission:** our principal namespace types — Principal, UserPrincipal, GroupPrincipal, ContainerPrincipal, SensitivityLabel.
- **Actions:** ActionType, ActionInvocation, ActionParameter, linking objects (subjects of actions) to functions (the executable logic).

13.2 Customer ontology

Domain-specific. An investment firm has Fund, LimitedPartner, CapitalCall, Distribution. A SaaS company has Account, OpportunityStage, Renewal, ExpansionEvent. A hospital has Patient, Encounter, LabResult. These are proposed by the ontology agents (§14), refined by humans, and committed to the customer's OWL repository.

```
# customer/acme/ontology.ttl
@prefix owl: <http://www.w3.org/2002/07/owl#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .
@prefix upper: <https://platform.internal/upper#> .
@prefix acme: <https://acme.com/ontology#> .

acme:Account a owl:Class ;
  rdfs:subClassOf upper:Organization ;
  rdfs:label "Account"@en ;
  rdfs:comment "A customer organization in our CRM."@en .

acme:hasOwner a owl:ObjectProperty ;
  rdfs:domain acme:Account ;
  rdfs:range upper:Person ;
  rdfs:label "has owner"@en .

acme:tier a owl:DatatypeProperty ;
  rdfs:domain acme:Account ;
  rdfs:range xsd:string .
```

13.3 SHACL shapes as the contract

Every customer class has SHACL shapes. Fact-mining agents (§15) produce facts only if they conform. Ontology agents (§14) propose shape changes when they see persistent violations. SHACL is executable schema, not a docstring.

```
@prefix sh: <http://www.w3.org/ns/shacl#> .
@prefix acme: <https://acme.com/ontology#> .

acme:AccountShape a sh:NodeShape ;
  sh:targetClass acme:Account ;
  sh:property [
    sh:path acme:hasName ;
    sh:datatype xsd:string ;
    sh:minCount 1 ;
    sh:maxCount 1 ;
  ] ;
  sh:property [
    sh:path acme:hasOwner ;
    sh:class upper:Person ;
    sh:minCount 1 ;
  ] ;
```



```
sh:property [
  sh:path acme:tier ;
  sh:in ( "enterprise" "midmarket" "smb" ) ;
] .
```

Initial shapes are generated automatically from the OWL ontology using Astrea or OWL2SHACL. Both are well-established tools in the semantic-web community. The ontology agents then refine them based on what they observe in the actual data.

14. Ontology agents

An ontology is dead the day it's written. Schemas evolve as the business does — new product lines, new lifecycle stages, new contract types. Ontology agents are autonomous workers that keep the customer ontology and its SHACL shapes in sync with reality, by continuously mining the actual data.

14.1 Three sub-roles

- **Class proposer.** Watches incoming episodes. When it sees a cluster of entities that don't fit any existing class but recur (e.g. "capital call" mentioned 40 times this month in finance documents), it proposes a new class with a candidate OWL definition and example instances.
- **Property refiner.** Watches facts being rejected by SHACL. If 30% of rejected facts have a specific property value not in the `sh:in` enumeration, it proposes a shape relaxation. Conversely, if 99.9% of facts in a property have integer values but the shape allows strings, it proposes tightening.
- **Ontology aligner.** Detects when two customer-defined classes are likely the same (e.g. "Customer" and "Account" used with the same property fingerprint) and proposes a merge or `owl:equivalentClass` alignment.

14.2 Workflow

```
OntologyAgent.run():
  episodes = bitemporal_kg.episodes_since(last_run)
  candidates = LLM.extract_class_candidates(episodes,
                                           current_ontology)

  for c in candidates:
    if frequency(c) > threshold and not_subsumed_by(c,
                                                    current_ontology):
      shapes = generate_shacl(c, observed_instances)
      proposal = OntologyProposal(
        class=c,
        parent=infer_parent(c, current_ontology),
        shapes=shapes,
        supporting_episodes=top_k_episodes(c),
        confidence=score(c))
      review_queue.put(proposal)

# Recent literature (LLMs4OL Challenge 2025, OntoEKG 2026)
# shows multi-agent decomposition outperforms single-agent
# extraction; we use one agent per sub-role + a coordinator.
# F1 ≈ 0.72 on the OntoEKG benchmark; we target 0.85+ on
# customer-specific data via in-context examples from the
# customer's own approved ontology.
```

14.3 Human-in-the-loop, Palantir-style branching

Ontology changes are governance events. They are never auto-applied. Palantir uses Branching and Proposals — we adopt the same model:

- Every proposal opens a Git PR against the customer's ontology repo.
- The PR includes the OWL diff, the SHACL diff, supporting episodes (2-10 examples), and a confidence score.
- Reviewers approve or reject; on approve, the change cascades — shapes recompile, fact-mining agents re-run on recent episodes under the new shapes, existing facts are revalidated.
- Branching means changes can be proposed and tested in isolation before merging to the main ontology — same pattern that engineering teams already understand from code.

15. Fact-mining agents

Where ontology agents shape the schema, fact-mining agents fill it. They run continuously, scanning new documents, conversations, calendar events, and CRM updates, and extracting structured facts that conform to the current SHACL shapes.

15.1 Per-episode pipeline

```
FactMiningAgent.process(episode):
# 1. ACL check – only mine from data the agent can see
if not authorized(agent.principal,
                  episode.allowed_principals):
    return # skip

# 2. Entity extraction with the customer's ontology
entities = LLM.extract_entities(
    episode, classes=ontology.classes,
    examples=ontology.examples)

# 3. Entity resolution (see §16)
entities = entity_resolver.resolve(entities)

# 4. Relation extraction
triples = LLM.extract_triples(
    episode, properties=ontology.properties,
    entities=entities)

# 5. Temporal extraction (the tricky one)
for t in triples:
    t.valid_from, t.valid_to = LLM.extract_validity(t, episode)
    # "Alice was promoted to VP last Tuesday" → resolve
    # "last Tuesday" relative to episode.timestamp

# 6. SHACL validation
result = shacl_engine.validate(triples, current_shapes)
rejected = result.failures
if rejected:
    feedback_loop.send(rejected) # informs property refiner

# 7. Supersession
for t in result.accepted:
    existing = kg.find_overlapping(t.subject, t.predicate,
                                  t.valid_from)
    if existing and conflicts(existing, t):
        kg.supersede(existing, t)
    else:
        kg.insert(t)

# 8. Provenance: every triple carries source_episode,
#     extracted_by (this agent + version), confidence
# 9. ACL: inherit allowed_principals from episode unless
#     ontology pins it (some classes have static ACLs)
```

15.2 Confidence and human review thresholds

Every fact carries a confidence score. Facts below a configurable threshold (default 0.7) go to a review queue instead of the live KG. The threshold is per-class:

```
# customer/acme/extraction_policy.yaml
thresholds:
  acme:Contract.hasParty:      0.95 # contract parties matter
  acme:Contract.hasEffectiveDate: 0.90
  acme:Account.hasOwner:      0.85
```

```
acme:Meeting.hasAttendee:      0.50  # easy and low-stakes
upper:Person.hasEmail:         0.70  # default
```

```
auto_approve_below: 0.0      # never auto-approve
auto_reject_below:  0.30     # always reject
```

15.3 Agents that mine company data

The platform ships several pre-built mining agents. Customers can add more via the SDK.

Agent	Source episodes	Facts produced
ConversationMiner	Slack, Teams, Zoom transcripts, email	Decisions, action items, attendees, sentiment, topic clusters
TicketMiner	Jira, Linear, ServiceNow, Zendesk	Issue lifecycle, ownership transitions, blockers, resolution patterns
CrmMiner	Salesforce, HubSpot CDC	Account changes, opportunity stage transitions, renewal signals, churn predictors
DocumentMiner	Drive, Confluence, Notion, SharePoint	Document-to-project links, authorship, decisions, deadlines
CodeMiner	GitHub, GitLab, Bitbucket	Service ownership, dependency edges, deployment events, incident-to-commit links
CalendarMiner	Google Calendar, Outlook	Meeting topics, attendee networks, recurring patterns, schedule changes
FinanceMiner	ERP CDC, expense systems	Spend by vendor, budget tracking, anomaly signals
HrMiner	Workday, BambooHR	Org structure, manager changes, employment events

15.4 Agents that just retrieve facts for their own decisions

A primary use case: agents that don't insert anything, they only retrieve. Our MCP server exposes the read surface; any agent running anywhere can connect.

Examples we expect to see in week one:

- **On-call agent.** Paged at 3am. Calls `list_facts_about(service:checkout-api)` and `search("recent incidents checkout")`. Drafts a runbook delta. Never opens our UI.
- **Sales agent in Cursor.** Engineer is writing a customer-specific integration. Cursor (via our MCP server) calls `get_fact(subject=acme:Account/123, predicate=acme:integrationConfig)`. Provides correct config inline.
- **Renewals agent.** Cron job. SPARQL query for accounts whose `acme:nextRenewalDate` is within 60 days AND whose `acme:healthScore` dropped > 15 points in the last 30 days. Posts to Slack with summary.
- **Compliance agent.** Runs at month-end. SPARQL for all decisions in `acme:HighRiskCategory` made without a written approval fact in the KG. Drafts the audit package.

The platform is the substrate; the agents are the applications. This is the Palantir AIP positioning, except agents can be built by anyone, hosted anywhere, and authenticated via standard OAuth.

16. Entity resolution

Without good entity resolution, the KG splinters. "Alice", "Alice Smith", "alice@acme.com", "Alice Smith (sales)", and u_8a9d2e all need to collapse to one node. Gartner estimates poor data quality (largely duplicates) costs the average enterprise \$13M annually. Children's Medical Center Dallas reduced patient duplicates from 22% to 0.14% with proper entity resolution. For an enterprise KG, this is infrastructure, not a feature.

16.1 Cascade matching

Three tiers, decreasing in volume, increasing in cost per call:

```
def resolve(rec_a, rec_b):
    # Tier 1: rules — ~40% of decisions, free
    if exact_id_match(rec_a, rec_b):
        return MATCH, 1.0
    if definite_non_match(rec_a, rec_b): # different orgs, etc.
        return NO_MATCH, 1.0

    # Tier 2: ML — ~55% of decisions, cheap
    ml_score = ditto_or_finetuned_model.predict(rec_a, rec_b)
    if ml_score > 0.9:
        return MATCH, ml_score
    if ml_score < 0.3:
        return NO_MATCH, 1 - ml_score

    # Tier 3: LLM — ~5% of decisions, edge cases only
    return llm_match(rec_a, rec_b)
```

16.2 Semantic blocking

Pairwise comparison is $O(n^2)$ and impractical at enterprise scale. We use embedding-based blocking: cluster candidate records by embedding similarity (top-50 nearest neighbors), then run the cascade only within each cluster. This brings the effective comparison count from n^2 to roughly $50n$. MERAI and similar production systems scale to 15M+ records this way.

16.3 The transitive-closure trap

Matching produces pairs; we need clusters. Naive transitive closure ($A=B, B=C \Rightarrow A=B=C$) is dangerous: a few false-positive pairs in a dense graph collapse everything into one giant supercluster. We use connected components with a confidence threshold edge: only edges above 0.85 contribute to closure, and we cap cluster size with a sanity check ("is it plausible that this person appears 47 times in our data?").

17. Provenance, lineage, audit (W3C PROV)

Every fact in the KG and every action executed by the action runner generates a PROV-O entry. PROV is the W3C recommendation for provenance; we use it because it's standard, machine-readable, and survives audits.

17.1 The PROV trinity

- **Entity.** A fact in the KG, a document, a chunk, an action result.
- **Activity.** An extraction run, an action execution, an ontology approval.
- **Agent.** A user, a service account, a fact-mining agent, an ontology agent.

Standard relations:

```

prov:wasGeneratedBy    : entity ← activity
prov:wasDerivedFrom    : entity ← entity
prov:wasAttributedTo   : entity ← agent
prov:wasAssociatedWith: activity ← agent
prov:used               : activity ← entity
prov:startedAtTime     : activity → xsd:dateTime
prov:endedAtTime       : activity → xsd:dateTime

```

17.2 Bi-temporal audit log

All permission-service changes are appended to a bi-temporal audit log, modeled the same way as facts in the KG. "Who had access to X at time Y" is a single SPARQL query.

```

PREFIX prov: <http://www.w3.org/ns/prov#>
PREFIX : <https://platform.internal/schema#>

# "Who had access to document D on 2026-02-15?"
SELECT ?user WHERE {
  ?perm a :PermissionGrant ;
        :resource doc:D ;
        :grantee ?user ;
        :valid_from ?vf ;
        :valid_to ?vt .
  FILTER(?vf <= "2026-02-15T00:00:00Z"^^xsd:dateTime)
  FILTER(?vt >= "2026-02-15T00:00:00Z"^^xsd:dateTime)
}

```

17.3 OpenLineage integration

For the ingestion pipeline, we also emit OpenLineage events. This lets customers see our pipeline alongside their dbt, Airflow, and Spark jobs in their existing lineage tools. OpenLineage is the operational counterpart to W3C PROV: PROV for the KG facts, OpenLineage for the pipeline.

18. Retrieval pipeline (hybrid)

Hybrid retrieval over three substrates: the vector index, the bitemporal KG (via SPARQL), and BM25 lexical search. The right substrate depends on the question:

- **Semantic similarity** ("find me docs about Q3 strategy") → vector.
- **Structured** ("who owns account 123 as of last March") → SPARQL with valid_at.
- **Lexical** ("find the contract with clause 4.2.1") → BM25.
- **Multi-hop** ("all renewals at risk because their CSM left") → SPARQL + vector hybrid.

18.1 Reference flow

```
def query(user, q):
    principals = principal_cache.load(user)
    intent      = classify_intent(q)

    candidates = []
    if intent.uses_vector:
        candidates += vectordb.search(
            embed(q), k=50,
            filter=allowed_principals_intersects(principals))

    if intent.uses_kg:
        sparql = nl_to_sparql(q, ontology)
        sparql = grammar_constrained_repair(sparql)
        candidates += kg.execute(sparql, principals=principals)

    if intent.uses_lexical:
        candidates += bm25.search(
            q, k=50,
            filter=allowed_principals_intersects(principals))

    # Reciprocal Rank Fusion across substrates
    fused = rrf(candidates)
    # Cross-encoder for top-50 only (expensive)
    reranked = cross_encoder.rerank(q, fused[:50])

    # Authoritative recheck (see §10, §11)
    approved = batch_acl_check(user, reranked)
    if tenant.high_sensitivity:
        approved = source_recheck(user, approved[:10])

    audit.write(user, q, approved, ts=now(),
                principals_hash=hash(principals))
    return llm.generate(q, ctx=approved[:10])
```

18.2 Vector index choice

Three viable choices in 2026. We default to pgvector for development and small-to-mid customers, and offer Qdrant or Turbopuffer at scale.

Choice	When
pgvector 0.9+	Default. Up to ~10M vectors, customer wants one fewer system to operate, ACID consistency with relational metadata. The industry's actual default in 2026.
Qdrant	Self-hosted preference, Rust performance, rich JSON-payload filtering for our metadata-heavy use case. Strong for the 10M-100M range.
Turbopuffer	S3-backed, no namespace limit, hybrid search built in even on low tiers. Best for multi-tenant SaaS where tenant isolation via namespaces matters.
Weaviate	If we lean heavier on built-in BM25 + vector hybrid. Considered.
Vespa	Billion-scale hybrid retrieval. Future option, overkill today.

18.3 Graph-distance reranker (from Zep)

For KG-heavy queries, results closer to a designated centroid entity get a boost. The centroid is usually the user themselves (personalization) or the entity the query is about. This is the Zep-paper episode-mentions reranker, applied to the customer KG.

19. Actions: the kinetic layer

Palantir's biggest architectural innovation, and what separates an operational platform from a search product. Objects are nouns; actions are verbs. A read-only knowledge graph is interesting; a knowledge graph paired with governed, audited, write-back actions is a substrate for AI agents that actually do work.

19.1 Action types

An action type is a declarative description of an operation: its input schema (JSON Schema), the principals allowed to execute it, the validation logic, the side effects, and the audit requirements. Every connector that supports writes contributes action types to the registry.

```
# Example action type declaration
ActionType:
  id: salesforce.update_opportunity_stage
  source: connector:salesforce
  description: |
    Move a Salesforce Opportunity to a new stage. Records a fact
    in the KG capturing who triggered, when, and prior state.
  input_schema:
    type: object
    properties:
      opportunity_id: { type: string }
      new_stage:
        type: string
        enum:
          ["Prospecting", "Qualification", "Proposal",
           "Negotiation", "Closed Won", "Closed Lost"]
      reason: { type: string, maxLength: 500 }
    required: [opportunity_id, new_stage]

  permission:
    required_principals:
      - role:salesforce/SalesUser
      OR
      - group:revops@acme.com

  validation:
    # Custom function (TypeScript / Python / WASM)
    - fn:validate_stage_transition
    - fn:require_close_reason_when_closed_lost

  approval:
    # Optional approval workflow for high-value opps
    when: "opportunity.amount > 500000"
    approvers: [role:salesforce/SalesManager]

  side_effects:
    - kind: source_writeback
      connector: salesforce
      operation: PATCH /subjects/Opportunity/{opportunity_id}
    - kind: kg_fact_emit
      class: acme:OpportunityStageTransition
    - kind: audit_log
    - kind: notify
      channel: slack:#revops
```

19.2 Idempotency and exactly-once semantics

Every action carries an idempotency key (client-provided or server-generated). Retried calls with the same key return the original result rather than executing twice. For source writes, this means we never accidentally create duplicate Jira tickets or update the same Salesforce record twice. Inspired by Stripe's API and Palantir's atomic-transaction model.

19.3 Approval workflows

High-stakes actions (financial, legal, anything tagged `label:restricted`) can require approval before execution. The approval queue is the same infrastructure as ontology proposals from §14 — proposal-and-review, with full provenance, comment threads, and PR-style approvals.

20. MCP integration

20.1 Outbound: the platform as an MCP server

The platform ships an MCP server. Any MCP-aware client connects with a single OAuth flow and gets the platform's capabilities as native tools. Not doing this in 2026 means being invisible to where developers and analysts actually live.

Tool surface (initial):

```
search(query, k=10, source_filter=None, valid_at=None)
    Hybrid retrieval; respects calling user's ACL automatically.

get_fact(subject, predicate=None, valid_at=None, as_of=None)
    Direct fact lookup, with bitemporal context.

query_sparql(query, valid_at=None, as_of=None)
    Raw SPARQL with automatic ACL post-filter.

list_facts_about(entity_iri, since=None)
    All facts where entity is subject or object.

time_travel_query(query, valid_at)
    Answer "as of date X".

propose_fact(subject, predicate, object,
             valid_from, valid_to, evidence)
    Write a fact (SHACL + ACL + review-queue gated).

execute_action(action_type, input, idempotency_key)
    Run a kinetic action; the user-facing trigger from agents.

list_proposals(status="pending")
approve_proposal(proposal_id, comment)
    Ontology evolution surface.

get_provenance(fact_id)
    Episodes, agent, confidence, timestamps.
```

20.2 Authentication

OAuth 2.1 with PKCE. The MCP client receives a bearer token bound to the end-user's enterprise identity. All tool calls execute with that identity — ACL filtering is automatic. No service-account shortcuts, no shared keys. Tokens are short-lived (1 hour) with refresh. This is the same pattern Supabase and other production MCP servers have converged on.

20.3 Cursor and Claude Code specifics

Both support stdio and HTTP/SSE MCP transports. We ship:

- **Remote HTTP/SSE server.** Primary deployment. Single endpoint, OAuth flow, scoped per workspace.
- **"Add to Cursor" / "Add to Claude Code" buttons.** One-click install from our documentation page.
- **Project-scoped config.** Docs show how to add the server to `.cursor/mcp.json` or Claude Code's MCP config for team-shared setup.
- **Local stdio shim.** A Node CLI wraps the remote API for offline / on-prem deployments. Useful for security-paranoid customers.

```
// .cursor/mcp.json (committed to the customer's repo)
{
  "mcpServers": {
    "acme-knowledge": {
      "type": "http",
      "url": "https://mcp.platform.example.com/v1",
      "auth": { "type": "oauth" }
    }
  }
}
```

20.4 Inbound: the platform as an MCP host

Where a source system itself ships an MCP server (Supabase, GitHub, Linear, Airbyte, etc.), our connector layer can consume it directly instead of writing a bespoke connector. We still need to extract ACLs separately — most source MCP servers don't expose permission data — but it shortens time-to-integration for the content channel.

21. Reference deployment topology

Four supported topologies. Customers choose based on data sensitivity, regulatory requirements, and operational appetite.

21.1 Topology A: Multi-tenant SaaS (default)

We host. Tenants are isolated by `tenant_id` prefix in all storage. Connectors run in our cloud, source credentials stored in our KMS, all data in our region of choice (US, EU, AU at GA).

- **Pros:** fastest onboarding (minutes), lowest TCO for the customer, fastest iteration on our side.
- **Cons:** some regulated industries (defense, certain HIPAA customers, EU sovereign cloud requirements) cannot use shared infrastructure.
- **Tenant isolation:** separate SpiceDB schemas per tenant, separate Kafka topics per tenant, separate RDF graphs per tenant. Vector index uses tenant-id as a hard filter at the engine level.

21.2 Topology B: Customer VPC (BYOC)

We deploy into the customer's cloud account (AWS, Azure, GCP) with our Terraform / Helm modules. The control plane stays in our cloud; the data plane (connectors, storage, query layer, agents) runs in the customer's VPC. Inspired by Airbyte Self-Managed Enterprise's BYOC option.

- **Pros:** data never leaves the customer's network. Satisfies most enterprise security reviews.
- **Cons:** slower onboarding (days, not minutes), heavier support burden. Customers prefer this — we should plan to support it from M5.

21.3 Topology C: Single-tenant managed SaaS

We host, but in a dedicated cluster per customer. Same code paths as multi-tenant SaaS, different operational profile. Useful for large customers who want SaaS economics but need physical isolation.

21.4 Topology D: Fully self-hosted (open core)

The core platform is open source under Apache 2.0. Customers run the whole thing on their own infrastructure, with no calls home. Commercial features (managed connectors, advanced agents, compliance dashboards, support) require a license but are optional. This is Onyx's model and it's the only way to win the most security-conscious customers — and to be honest about "available to everyone".

22. Security model

22.1 Threat model

- **Untrusted query user.** Cannot see content they couldn't see in source.
- **Compromised user credentials.** Blast radius limited to what that user could access. No privilege escalation through us.

- **Malicious tenant admin.** Can grant themselves access in our system, but the source recheck for high-sensitivity tenants prevents bypassing source ACLs.
- **Compromised connector credentials.** Rotated regularly, scoped narrowly, stored in HSM-backed KMS. Per-tenant credentials never reused.
- **Prompt injection from indexed content.** Mitigated by treating all indexed content as untrusted, requiring citations for all generated claims, and never auto-executing tool calls that originated from indexed-content suggestions.
- **Insider threat at us.** Customer-managed encryption keys (CMEK) so even our SRE cannot decrypt customer data. Audit log of every access by our staff. SOC 2 Type 2 + ISO 27001 from M9.

22.2 Defenses by failure mode

Failure mode	Defense
Stale group membership	Group-change events invalidate principal cache directly; 60s TTL backstop.
Stale document ACL	SpiceDB recheck on top-K; source recheck for high-sensitivity tenants on top-N.
Information leakage via result count	Always backfill from oversampled candidates; never return short result sets silently.
Leakage via LLM synthesis	ACL filter at context assembly, before any model call. No post-hoc redaction.
New-enemy problem	SpiceDB ZedTokens for read-after-write consistency during sensitive sync windows.
Prompt injection	Untrusted-content treatment; citation requirement; tool-call quarantine for indexed-content suggestions.
Oversharing	Sensitivity-label policy (§11.4), access-pattern analytics for admins.
Tenant-isolation breach	Tenant ID as a hard filter at every storage layer; per-tenant encryption keys; quarterly red-team exercises.

23. Performance, scale, SLAs

23.1 Initial scale targets

Documents	50M (per tenant)
Facts	100M (per tenant)
Permission relationships	10M (per tenant)
Users	100K (per tenant)
Groups	10K (per tenant)
Sources per tenant	20-30 at GA
Concurrent users	5K (per tenant)
Sustained QPS	100 (per tenant)
Burst QPS	1K (per tenant)

23.2 Latency budget

Stage	P50	P95	P99
Principal expansion (cached)	5ms	15ms	40ms
Vector retrieval (k=50)	30ms	80ms	180ms
SPARQL retrieval (typical)	40ms	120ms	300ms
BM25 retrieval	20ms	50ms	100ms
RRF fusion + reranker	60ms	150ms	300ms
SpiceDB batch_check (50 items)	10ms	30ms	80ms
Source recheck (top-3, optional)	200ms	500ms	1500ms
LLM generation (excluded from us)	—	—	—
End-to-end (no source recheck)	200ms	500ms	1100ms

23.3 Freshness SLAs

- **Webhook-backed sources:** $P95 \leq 30$ seconds from source event to indexed.
- **CDC-backed sources:** $P95 \leq 60$ seconds.
- **Poll-backed sources:** bounded by configured poll interval (typically 5-15 minutes).
- **ACL changes:** $P95 \leq 5$ minutes; hard ceiling 1 hour, backstopped by source recheck for high-sensitivity tenants.
- **Ontology approval-to-effect:** ≤ 5 minutes (shape recompile + validator restart).

24. Build vs. buy decisions

Where we write code, where we adopt open source, where we license. Critical for keeping the engineering org small without compromising the platform.

Component	Decision	Rationale
Connector framework	Build	Our unique value. SDK contract + permission model.
Managed connectors (top 10)	Build	Quality and ACL fidelity are the core moat.
Long-tail connectors	Adopt OSS (Airbyte-format compatible)	Reuse 600+ Airbyte / Singer connectors for content; pair with custom ACL extraction.
CDC infrastructure	Adopt Debezium	Industry standard, mature, supports every major DB.
Event bus	Adopt (Kafka or Redpanda)	Not our differentiator.
Authorization engine	Adopt SpiceDB	Most mature open-source Zanzibar implementation.
Vector DB	Adopt (pgvector default, Qdrant at scale)	Commodity; pick best of breed.
RDF triple store	Adopt (GraphDB Free dev, Stardog or Jena Fuseki prod)	Standards compliance matters more than implementation.
SHACL validator	Adopt (TopBraid SHACL, pySHACL)	Mature OSS.
Ontology IDE for customers	Adopt (Protégé compatibility) + build a web UI on top	Web UI for casual users; power users use Protégé.
Entity resolution	Build (cascade) + adopt base models	ML core can use OSS (Ditto, DeepMatcher); cascade and blocking are ours.
LLM(s) for extraction	Adopt (multi-provider abstraction)	Don't pick one; ride the cost / quality curve. Anthropic, OpenAI, Google, Mistral, plus local Qwen 3 / Llama 4 for private deploys.
MCP server	Build	The shape of our tool surface is our product.
Audit log + lineage	Build (PROV-O) + adopt (OpenLineage)	PROV in the graph; OpenLineage for the pipeline.
Identity provider integration	Adopt (OAuth 2.1 + OIDC + SCIM via standard libs)	Commodity.
Observability	Adopt (OpenTelemetry, Prometheus, Grafana)	Standard.

25. Milestones

Twelve phases, each with a clear shippable deliverable. The first six are foundation; the next six are product expansion.

Phase	Deliverable	Notes
M1	Permission service + first connector	SpiceDB schema, materialized ACL, principal expansion cache. Google Drive end-to-end with changes.list.
M2	Vector retrieval with ACL filter	pgvector, oversampled retrieval, SpiceDB recheck, backfill loop, audit log. Parity with v1 scope.
M3	Bitemporal KG core	GraphDB committed for dev. Ingestion writes facts with valid + transaction time. Internal SPARQL endpoint.
M4	Upper ontology + SHACL validation	Ship Person / Document / Conversation / Project / Issue / Email vocab. SHACL on insert. Astrea-generated initial shapes.
M5	Connector breadth — top 8	Confluence, Slack, Notion, SharePoint, Jira, Salesforce, GitHub, Gmail. Common SDK proven in production.
M6	MCP server v1	search, get_fact, query_sparql, time_travel_query. OAuth, remote HTTP/SSE. Cursor and Claude Code documented.
M7	Fact-mining agents v1	ConversationMiner, DocumentMiner, CrmMiner. Confidence scoring. Review queue.
M8	Ontology agents v1	Class proposer + property refiner. Review UI. Per-customer OWL in Git.
M9	Hybrid retrieval + community subgraph	Vector + SPARQL + BM25 fusion. Community detection + summaries. Graph-distance reranker.
M10	Layered access control	Row + column + cell policies. Sensitivity labels. Policy hooks at context assembly. Source recheck mode generally available.
M11	Action runner / kinetic layer	Write-back actions. Idempotency. Approval workflows. Initial action registry for top 5 sources.
M12	Self-hosted distribution + open-source release	Apache 2.0 core. Helm charts. Terraform modules. Quickstart for BYOC and fully-self-hosted deployments.

26. Open questions

- **RDF store at scale.** GraphDB Free for dev is settled. Stardog vs. Jena Fuseki vs. self-hosted Virtuoso for production needs a spike under realistic load (100M triples, 100 QPS).
- **Property-graph projection.** If SPARQL performance becomes problematic for specific hot queries, do we maintain a Neo4j projection? Cost: operational complexity. Benefit: 10x latency on hot paths.
- **MCP server pricing.** Free for individuals, metered for enterprise, or bundled? Affects M6 scope and our positioning vs. Glean, Dust.
- **SPARQL exposure to customers.** Power users will want it; security-conscious customers may not. Default off, opt-in per tenant?

- **Ontology-agent autonomy.** Which classes of changes can auto-apply (e.g. shape relaxations below threshold), vs. always human-reviewed?
- **Fact-mining budget.** Each customer has a finite token budget for ongoing extraction. Default and metering?
- **Multi-tenancy isolation.** Separate SpiceDB instances per tenant or shared with strict prefixes? Glean isolates; we should probably too.
- **Distillation.** Can we fine-tune small models (Qwen 3, Llama 4) on customer-specific extraction once we have enough labeled data? Reduces extraction cost by 10x at the price of model-management complexity.
- **Bring-your-own-LLM.** Customers want to choose. Glean ships a 15-LLM hub; what's the right v1 set for us? Anthropic, OpenAI, Google, plus one local? Plus Mistral for EU customers?
- **Action registry.** Curated vs. customer-extensible? Probably both — curated for connector-provided actions, customer-extensible via SDK.

27. References

Foundational papers and standards

- Pang et al., *Zanzibar: Google's Consistent, Global Authorization System*, USENIX ATC 2019.
- Rasmussen et al., *Zep: A Temporal Knowledge Graph Architecture for Agent Memory*, 2025 (arXiv:2501.13956).
- Microsoft Research, *From Local to Global: A Graph RAG Approach to Query-Focused Summarization*, 2024.
- Oyewale, *LLM-Driven Ontology Construction for Enterprise Knowledge Graphs (OntoEKG)*, Liber AI Research, 2026.
- Cimmino et al., *Astrea: Automatic Generation of SHACL Shapes from Ontologies*, 2020.
- *On the Implementations of the BiTemporal RDF Model*, Informatics, 2026.
- Anselma et al., *Bitemporal Property Graphs*, ADBIS 2026.
- Ranganath, *PIPE-RDF: LLM-Assisted Pipeline for Enterprise RDF Benchmarking*, UC San Diego.
- *HKUDS LightRAG: Simple and Fast Retrieval-Augmented Generation*, EMNLP 2025.
- Belmabrouk et al., *A Multi-LLM Agent System for Modular Ontology Population*, 2026.
- *MERA1: Robust and Efficient Pipeline for Enterprise-Level Large-Scale Entity Resolution*, 2025.
- W3C: RDF 1.1, OWL 2, SHACL, SPARQL 1.1, PROV-O.
- *Model Context Protocol Specification*, modelcontextprotocol.io.
- *OpenLineage Specification*, LF AI & Data Foundation.

Product references (architectural inspiration)

- Palantir Foundry: Ontology Architecture docs; "The Ontology System" (Architecture Center); "Connecting AI to Decisions with the Palantir Ontology" (Palantir Blog, Nov 2025).

- Glean: Connectors documentation; Permissions and ACLs guide; Indexing API documentation; Futurum Group (April 2026) on Glean's \$200M ARR; TechCrunch (Feb 2026) "The enterprise AI land grab".
- Microsoft Fabric / OneLake: Security overview; OneLake Security public preview docs; Microsoft Purview integration.
- Snowflake / Databricks / Redshift / BigQuery: row-level and column-level access control documentation.
- Airbyte, Nango, Fivetran, Hotglue: connector taxonomy and tiering models.
- Debezium: CDC architecture and connector library.
- SpiceDB (Authzed) and OpenFGA: Zanzibar-derived authorization engines; ZedTokens for read-after-write consistency.
- Pinecone, Qdrant, Weaviate, Turbopuffer, LanceDB: 2026 vector database comparisons.
- Cursor and Claude Code MCP documentation; Supabase MCP server as a reference for production MCP architecture.

Inspiration we've earned the right to cite

Palantir for the ontology shape. Glean for ACL discipline and the MCP server/host duality. Zep / Graphiti for bitemporality. Microsoft Fabric for the authorized-engine model. Foundry's branch-and-proposal pattern for ontology evolution. Stripe for idempotency. Stripe for documentation, frankly.

This is a living document. Comment threads open in Notion. Reviewable diffs in Git:

platform/rfcs/001-permission-aware-enterprise-kg.md. Substantive changes require maintainer review; typos and formatting may be merged on green CI.